Digital Agents Meet World Models: A Survey

Yuhan Chen[*] *chenyuhan5@xiaomi.com*
Pengzhi Gao[*] *gaopengzhi@xiaomi.com*
Qinzhuo Wu *wuqinzhuo@xiaomi.com*
Kun Huang *huangkun11@xiaomi.com*
Yike Liu *liuyike3@xiaomi.com*
Heng Qu *quheng@whu.edu.cn*
Kun Shao[†] *shaokun@xiaomi.com*
Jian Luan[†] *luanjian@xiaomi.com*

Abstract

Digital agents operating in software-defined environments, including games, web interfaces, mobile GUIs, tool-use systems, and code repositories, often benefit from anticipating the consequences of actions before execution. World models provide a principled mechanism for such anticipation, but their utility in digital settings is neither automatic nor uniform. Unlike physical environments, digital transitions are interface-mediated and program-governed, and the value of prediction depends less on perceptual realism than on preserving functionally relevant consequences for downstream decisions. This survey presents the first dedicated agent-centric treatment of world models across major digital-agent domains. We introduce a unified design space, $\mathcal{W} = (\mathcal{X}, \mathcal{L}, \mathcal{U})$, organized around three complementary questions: what is modeled, how predictive capability is built, and how world models are used inside the agent loop. Through this lens, we synthesize a rapidly growing literature spanning semantic transition models, visual and latent simulators, tool-state predictors, executable code-based world models, and hybrid predictive systems. We further examine environments, datasets, and evaluation protocols as a coupled empirical pipeline, and highlight open challenges including the persistent gap between predictive fidelity and downstream decision utility.

Repository: `https://github.com/Darwin-Agent/awesome-world-models-for-digital-agents`

1 Introduction

Digital agents, autonomous systems that act in software-defined environments, have advanced rapidly with the rise of large language models (LLMs), vision-language models (VLMs), and reinforcement learning. These environments span games (Silver et al., 2016; Vinyals et al., 2019), web and graphical user interfaces (GUIs) (Liu et al., 2024a; Qin et al., 2025; Team et al., 2026), tool-use ecosystems (Qin et al., 2024), and code repositories (Li et al., 2022; Rozière et al., 2024; Yang et al., 2024). Yet many agents still operate reactively, mapping observations directly to actions with limited modeling of how the environment will evolve. Such reactive behavior is especially costly in digital settings, where actions are easy to specify but hard to recover from: a web agent may navigate away from a multi-step workflow, a tool-use agent may trigger cascading failures, and a code agent may corrupt repository state.

World models provide a principled way to endow agents with anticipatory capability by learning action-conditioned dynamics (Ha & Schmidhuber, 2018; Hafner et al., 2020; Schrittwieser et al., 2020; Hafner et al., 2025). Motivated by classical successes in control, game playing, and robotics, recent work has begun exploring analogous ideas across games (Bruce et al., 2024), web and GUI interaction (Gu et al., 2025; Chae et al., 2025; Luo et al., 2025a), tool-use systems (Wang et al., 2025b; Xiao et al., 2026), and code environments (Tang et al., 2024; Dainese et al., 2024). At the same time, world models are not a universal remedy: learned dynamics can suffer from compounding errors, agents may exploit model inaccuracies, and

[*]Equal contribution.
[†]Corresponding author.

Table 1: Comparison with representative related surveys along domain coverage and analytical framing.

Survey	WM as Core	Agent-Centric	Games	GUI/Web	Tool Use	Code
Wang et al. (2024)				✓	✓	
Zhu et al. (2024)	✓		✓			
Ding et al. (2025)	✓		✓			
Feng et al. (2025b)	✓					
Tu et al. (2025)	✓					
Zhang et al. (2025)	✓					
Li et al. (2025b)	✓					
Long et al. (2025)	✓					
Kong et al. (2025a)	✓					
Bai et al. (2025)	✓		✓			
Dong et al. (2026)	✓		✓	✓		
Tan et al. (2026)	✓					
Chu et al. (2026)	✓	✓	✓	✓		
Ours	✓	✓	✓	✓	✓	✓

the cost of repeated model querying can outweigh its benefits when simpler reactive policies suffice (massoud Farahmand, 2018; Lambert et al., 2020).

Yet world models for digital agents are not simply extensions of the physical-world paradigm. Digital environments are interface-mediated, program-governed, and judged primarily by functional correctness rather than perceptual realism. Accordingly, usefulness depends less on raw predictive fidelity than on whether predictions preserve the action-relevant consequences the agent needs. The design space is correspondingly broad: useful digital world models span diverse architectures and serve different roles inside the agent loop, from decision-time planning to training-time data generation to predictive verification (Guan et al., 2026; Chen et al., 2026; Ding et al., 2026). The central question is therefore not only whether a model predicts the future, but which future it exposes, in what form, and for what downstream purpose.

Our scope covers agents operating in inherently digital environments, including games, web and GUI interaction, tool-use systems, and code generation. The inclusion criterion is whether the task's governing transition dynamics are fundamentally software-mediated, rather than the rendering medium. We accordingly exclude settings whose underlying task is primarily physical or embodied even when instantiated in digital simulators (Todorov et al., 2012; Dosovitskiy et al., 2017). Our aim is to understand how predictive modeling serves digital agent design, not to survey world models in general. Despite the growing importance of this topic, no existing survey provides a dedicated, agent-centric treatment of world models across these digital-agent domains (Table 1). We argue for an agent-centric lens. Existing discussions are often model-centric, organizing methods by architecture or prediction target, but the practical value of a world model depends equally on how it is trained, coupled to other components, and used to improve downstream behavior. We therefore introduce a unified design space $\mathcal{W} = (\mathcal{X}, \mathcal{L}, \mathcal{U})$, organized around what is modeled ($\mathcal{X}$), how predictive capability is built ($\mathcal{L}$), and how world models are used ($\mathcal{U}$). The framework lets us compare architecturally similar methods that serve different agent-facing purposes, and relate methods across domains that fulfill analogous predictive roles.

Table 1 makes this positioning explicit, comparing prior work along two dimensions: whether world models are treated as a core object of analysis and whether they are organized through an agent-centric lens, in addition to the digital-agent domains covered. Prior surveys either focus on physical and embodied settings (Feng et al., 2025b; Tu et al., 2025; Zhang et al., 2025; Li et al., 2025b; Long et al., 2025; Kong et al., 2025a), study world models at a broader methodological level with limited digital-agent coverage (Zhu et al., 2024; Ding et al., 2025; Bai et al., 2025), or survey agent capabilities without treating world models as an explicit design component (Wang et al., 2024; Dong et al., 2026). The closest concurrent work, Chu et al. (2026), shares our agent-centric stance and treats world models as a core subject, but covers digital-agent domains as part of a broader four-regime account without dedicated treatment of tool-use and code world models. We argue that closing this gap requires not just broader coverage but a unified framework connecting prediction, realization, and agent usage.

Our main contributions are as follows:

- We introduce a unified agent-centric design space $\mathcal{W} = (\mathcal{X}, \mathcal{L}, \mathcal{U})$ that organizes world models along what is modeled, how predictive capability is built, and how it is used, providing a common language across games, web and GUI, tool-use, and code domains.
- We use this framework to synthesize a rapidly growing literature, analyzing five architectural paradigms (LLM-based, visual generative, latent dynamics, code-based, and hybrid), three usage patterns (decision-time planning, training-time interaction, and predictive verification), and recurring construction-time challenges.
- We survey environments, datasets, and evaluation as a coupled empirical pipeline, arguing that they should be jointly assessed by whether they expose decision-relevant predictive structure.
- We identify key open challenges, including the gap between predictive fidelity and decision utility, data bottlenecks for action-conditioned modeling, environment drift, and the relationship between explicit world models and foundation models.

The remainder of this paper is organized as follows. Section 2 revisits classical formulations and explains why a model-centric view is insufficient. Section 3 introduces the design space. Sections 4 and 5 instantiate the construction ($\mathcal{X}$, $\mathcal{L}$) and usage ($\mathcal{U}$) dimensions, respectively. Section 6 surveys environments, datasets, and evaluation. Section 7 discusses open challenges.

2 Foundations of World Models in Digital Agents

This section establishes the conceptual foundation for the agent-centric perspective introduced above. We revisit the classical formulation of world models, formalize digital agents as a partially observable regime, and explain why a purely model-centric view must be extended for digital-agent settings.

2.1 Classical Formulation of World Models

A world model is commonly understood as a learned model of environment dynamics that allows an agent to predict the future consequences of its actions without executing them in the real environment. In its classical form, a world model is often written as

$$\mathcal{M} = (f, g, r), \tag{1}$$

where f is a transition model, g is an observation model, and r is a reward model.

The transition model characterizes how the latent environment state evolves after the agent takes an action. Specifically, let $s_t \in \mathcal{S}$ denote the latent state at time step t, $a_t \in \mathcal{A}$ denote the action selected by the agent, and s_{t+1} denote the resulting next state. A stochastic transition model parameterized by θ can then be written as

$$s_{t+1} \sim f_\theta(\cdot \mid s_t, a_t), \tag{2}$$

where $f_\theta(\cdot \mid s_t, a_t)$ defines a conditional distribution over possible next states given the current state-action pair. The observation model maps latent states to observations, which is especially important in latent world models where reconstruction provides a learning signal. The reward model estimates scalar feedback for planning or policy optimization. Together, these components support imagined rollouts, in which the agent internally simulates trajectories to guide decision making.

This formulation has been highly influential in model-based reinforcement learning and related areas because it cleanly separates dynamics prediction, observation generation, and reward estimation. It also captures the central intuition behind world models: an agent can improve behavior by reasoning over predicted futures rather than relying solely on trial-and-error interaction. In many classical settings, this formulation is effective precisely because the predictive object is relatively well defined: the world model approximates the environment dynamics sufficiently well for planning, control, or policy learning. However, this classical

view is fundamentally model-centric: it centers on predicting future states, observations, or rewards, but leaves open what should be predicted in a given setting, how predictive capability should be embedded in a larger agent system, and whether accurate predictions actually improve downstream behavior. As we argue below, these open questions become especially pressing for digital agents.

2.2 Digital Agents as Partially Observable Interaction Processes

We adopt a POMDP-style notation as a general description of the interaction loop, where the agent selects actions based on observations exposed through an interface rather than direct access to the complete underlying environment state. Formally, an environment can be described by the tuple

$$(\mathcal{S}, \mathcal{A}, \mathcal{O}, T, \Omega, R, \gamma),$$

where $\mathcal{S}$ denotes the underlying environment state space, $\mathcal{A}$ the action space, and $\mathcal{O}$ the observation space available to the agent. The transition function $T(s_{t+1} \mid s_t, a_t)$ specifies how the underlying state evolves after action a_t, the observation function $\Omega(o_t \mid s_t)$ maps underlying states to observable interface outputs, $R(s_t, a_t)$ denotes the reward function, and $\gamma \in [0, 1)$ is the discount factor. At each step, the agent receives an observation $o_t \in \mathcal{O}$ and selects an action according to a history-conditioned policy

$$\pi(a_t \mid h_t), \quad h_t = (o_{\leq t}, a_{<t}),$$

where h_t denotes the interaction history up to time t. Fully observable MDPs can be viewed as a special case in which the observation o_t is a sufficient representation of the underlying state for decision making.

This formalization is useful for digital agents because they often interact with software systems through restricted interface-level observations rather than complete internal state access. A game agent may receive pixels or symbolic descriptions rather than simulator memory; a web agent observes a rendered page rather than server-side state; a tool-use agent sees returned outputs rather than the complete external database; and a code agent observes files, tests, or execution traces rather than all latent dependencies and runtime states. The key point is therefore not that digital agents are uniquely partially observable, since many physical agents also face partial observability, but that digital environments expose software-governed state through task-dependent interfaces.

For world modeling, this distinction matters because useful prediction need not reconstruct the complete underlying state. Instead, a world model should capture the aspects of hidden, delayed, or externally stored state that affect future observations, action feasibility, rewards, or task success. In this sense, the relevant modeling target is often a task-relevant state abstraction underlying observable interface changes, rather than a pixel-level prediction or a fully specified simulator state.

2.3 Why Classical Formulations Are Insufficient

The classical formulation remains an essential starting point, but digital-agent settings make three limitations particularly salient.

First, the predictive target is no longer fixed a priori. The classical formulation assumes relatively well-defined objects of prediction: future states, observations, and rewards. In digital-agent settings, the useful prediction target is itself a design choice. A web agent may need to predict whether a click submits a form or modifies server-side state. A code agent may care less about the next textual observation than about whether a patch compiles and passes tests. The central modeling question is therefore not only how to approximate f, g, or r, but what consequence of action should be represented in the first place.

Second, world models in digital agents are rarely used as isolated predictors. Their practical value often depends on how they are embedded into larger systems that include perception, retrieval, reasoning, verification, and execution. A predicted GUI state may only become useful when grounded by screen parsing and action constraints. A code-oriented world model often gains value only when paired with execution feedback or static analysis. The world model is therefore less a standalone predictor than a component within a broader agent pipeline.

Third, predictive accuracy alone is not a sufficient objective. In digital settings, local fidelity may fail to translate into downstream utility when models miss sparse but decision-critical distinctions. A model that accurately predicts most pixels of a webpage can still be useless if it misses whether a button is disabled. A code model may generate plausible execution outcomes while overlooking a failing edge-case test. What matters is therefore not only the fidelity of f, g, or r as predictors, but whether their predictions preserve the distinctions that affect action selection and task success.

Viewed through the classical decomposition $\mathcal{M} = (f, g, r)$, these limitations show that the challenge is not merely to improve transition, observation, or reward prediction in isolation. Instead, digital-agent world models must specify what action consequence is worth modeling, how the predictive capability is grounded within a broader agent system, and how the prediction contributes to downstream decision making. These considerations motivate the shift developed in the next section: from a model-centric view of prediction to an agent-centric view of predictive utility.

2.4 Relationship to Related Concepts and Scope

The term world model overlaps with several neighboring concepts in the literature, including environment models, simulators, and foundation models. Clarifying these relationships is important not only for terminology, but also for defining the scope of this survey. In digital-agent settings, the boundaries among these concepts are often less clear than in classical model-based reinforcement learning, because many systems combine learned predictors, executable environments, and large pretrained models in a single agent loop.

World model vs. environment model Environment models or forward models typically refer to predictors of state transitions, such as $f(s_t, a_t)$. World models generally subsume this notion by including richer latent representations, observation generation, reward prediction, or other mechanisms needed for internal simulation. In digital-agent settings, many recent systems go further still, modeling interface deltas, executable outcomes, or task-level progress. Accordingly, we use the term world model in a broad but agent-relevant sense: it includes any action-conditioned predictive structure that exposes useful future consequences for planning, learning, or reasoning.

World model vs. simulator A simulator is usually a hand-engineered system that executes environment dynamics directly, often with high fidelity and deterministic behavior. A world model, by contrast, is typically learned from data and serves as an approximate, often probabilistic model of environment behavior. In digital environments, however, this boundary blurs: executable website environments, code-based sandboxes, and synthetic interface generators may all play world-model-like roles even when they are not learned in the classical sense. We include such systems when they provide action-conditioned surrogate dynamics for agent training or planning, but exclude static knowledge structures such as crawled site maps or API documentation that inform reasoning without modeling transition consequences.

World model vs. foundation model Large language models and vision-language models often exhibit implicit predictive knowledge about how digital environments work and may therefore function as implicit world models in some settings. However, such models are usually not trained or evaluated as explicit environment models, and they may struggle with calibrated multi-step consistency, controllability, or uncertainty estimation. For tasks requiring long-horizon planning, safety-aware verification, or calibrated uncertainty over branching futures, explicit world models are most likely to add value when they provide controllable, action-conditioned, and testable predictive structure. Indeed, recent evidence suggests that current agents often struggle to effectively leverage explicit world models even when available (Qian et al., 2026), indicating the bottleneck may lie in system-level coupling rather than predictive quality alone. In this survey, we treat foundation models as related but not identical to world models: they are included when used to realize action-conditioned predictive functions inside the agent loop, but not when they serve only as general-purpose reasoning backbones.

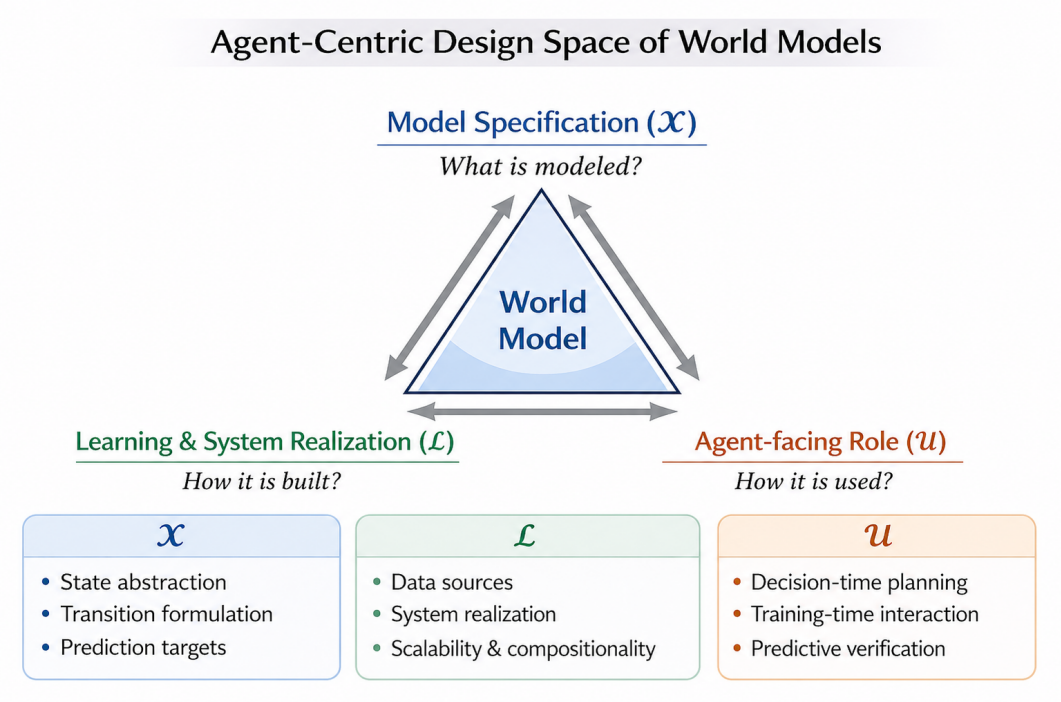

Figure 1: Illustrative view of the unified agent-centric design space, connecting model specification (*what is modeled*), learning and system realization (*how it is built*), and agent-facing usage (*how it is used*).

3 A Unified Agent-Centric Design Space of World Models

As argued in Section 2, existing discussions often conflate what is predicted, how predictive capability is built, and how it is used. To disentangle these distinct questions, we organize the field around a unified agent-centric design space:

$$\mathcal{W} = (\mathcal{X}, \mathcal{L}, \mathcal{U}), \tag{3}$$

where $\mathcal{X}$, $\mathcal{L}$, and $\mathcal{U}$ answer three complementary but conceptually distinct questions:

- $\mathcal{X}$ denotes **model specification** (*what is modeled*): the predictive interface exposed by the world model, covering state abstraction, transition formulation, and prediction target.
- $\mathcal{L}$ denotes **learning and system realization** (*how it is built*): the pathway through which predictive capability is learned, operationalized, and embedded into a usable system.
- $\mathcal{U}$ denotes **agent-facing role** (*how it is used*): the function the world model serves inside the agent loop, such as planning, training-time interaction, or predictive verification.

This distinction matters because many apparent differences in the literature arise from mixing these levels: predicting a future screenshot versus a semantic UI delta is a difference in $\mathcal{X}$; learning from offline logs versus online interaction is a difference in $\mathcal{L}$; and using the model for one-step evaluation versus synthetic trajectory generation is a difference in $\mathcal{U}$. Under this view, world models are best understood not as isolated predictors but as agent-facing predictive infrastructures whose usefulness depends on the alignment among all three dimensions. Figure 1 summarizes the design space. We elaborate each dimension in turn below.

3.1 Model Specification ($\mathcal{X}$)

The dimension $\mathcal{X}$ characterizes **model specification**, that is, **what is modeled**. It defines the predictive interface exposed to the rest of the agent. In digital environments, this dimension is especially important

because the underlying state is rarely observed directly. Instead, the agent operates over interface-level views that may be incomplete, redundant, heterogeneous, or only loosely aligned with the latent program state.

State abstraction A world model must first decide what representation of environment state it operates over. Possible abstractions range from raw observations, such as screenshots, HTML, text, or tool outputs, to more structured or compressed forms such as DOM trees, accessibility hierarchies, semantic graphs, execution traces, or learned latent embeddings. Raw observations preserve broad information but may obscure causal structure, while structured or latent abstractions can suppress irrelevant variation and expose the parts of state most relevant to downstream decisions (Hafner et al., 2019; Micheli et al., 2023).

Transition formulation A world model must also decide how future change is represented. It may predict direct next observations, transitions in latent or structured state space, semantic state updates, executable artifacts, or higher-level consequences of actions. This choice is especially consequential in digital environments, where action effects are often better described as discrete semantic or executable changes than as purely perceptual evolution. In many cases, what matters is not simply what the next screen looks like, but what changed in a way that affects planning, verification, or recovery.

Prediction target The predictive target can take many forms, including future observations, latent states, reward-related signals, interface deltas, executable intermediates, or higher-level outcomes such as milestone completion or execution success. Different targets expose different forms of foresight. Predicting a screenshot may support visually grounded acting, while predicting a structured delta or execution result may be more useful for search, verification, or counterfactual analysis (Chae et al., 2025; Mei et al., 2026). For example, WebWM (Feng et al., 2025a) predicts semantic HTML deltas (a state-delta target), while ViMo (Luo et al., 2025a) generates full next-screen observations (a full-observation target).

3.2 Learning and System Realization ($\mathcal{L}$)

The dimension $\mathcal{L}$ characterizes **learning and system realization**, that is, **how it is built**. It describes how the predictive capability specified by $\mathcal{X}$ is learned, grounded, and made operational in practice. This includes not only training data and optimization objectives, but also the system pathway through which a predictive design becomes a usable component of an agent architecture.

Data sources World models may be trained from offline logs, online interaction, or synthetic data. Offline trajectories provide realistic supervision over observed transitions; online interaction can expand coverage into underrepresented states, failures, and recovery behaviors; and synthetic trajectories or generated environments can provide scalable supervision when real interaction is expensive or risky (Hafner et al., 2020; Ye et al., 2021). In digital settings, where action-conditioned transition data is often scarce, domain-specific, and weakly covers rare branches, the provenance and structure of training data are especially important.

Learning objectives Training may be driven by supervised transition prediction, self-supervised latent modeling, reinforcement learning, execution-based feedback, verifier signals, preference-based optimization, or hybrid objectives. Importantly, these objectives need not optimize predictive fidelity alone. In digital-agent settings, they may also optimize functional validity, rollout consistency, executability, or usefulness for downstream planning and interaction (Ding et al., 2025), substantially broadening the design space beyond the classical next-state prediction paradigm.

System realization Even when two methods expose similar predictive targets, they may differ sharply in how they are operationalized. Some world models are lightweight predictors that can be queried repeatedly at inference time; others are tightly coupled to retrieval systems, rendering engines, execution environments, or external verifiers; still others serve as larger surrogate systems that support interactive rollout. These distinctions matter because they determine the actual cost, robustness, and reliability of using the model inside a practical agent loop.

Scalability and compositionality Digital-agent world models often need to operate over multiple modalities, multiple supervision sources, or multiple predictive modules. As a result, practical systems are frequently modular, retrieval-augmented, verifier-assisted, or jointly trained with planners and decoders. These considerations belong to $\mathcal{L}$ because they determine whether a predictive design can be realized efficiently and robustly at scale (Ge et al., 2024; Zheng et al., 2026).

Table 2: Representative methods mapped to $(\mathcal{X}, \mathcal{L}, \mathcal{U})$ coordinates. "Training" in the usage column refers to downstream policy training with model-generated experience, not world-model training.

Method	Domain	$\mathcal{X}$: Prediction Target	$\mathcal{L}$: Modeling Paradigm	$\mathcal{U}$: Usage
MuZero (Schrittwieser et al., 2020)	Game / board, Atari	Latent transition (reward/value/policy)	Latent dynamics	Planning (MCTS)
EfficientZero (Ye et al., 2021)	Game / Atari	Latent transition (reward/value)	Latent dynamics	Planning (MCTS), training
DreamerV3 (Hafner et al., 2025)	Game / diverse RL	Latent transition (state/reward)	Latent dynamics	Training (imagined rollouts)
IRIS (Micheli et al., 2023)	Game / Atari	Full observation (tokens)	Latent dynamics	Training (imagined trajectories)
Genie (Bruce et al., 2024)	Game / 2D platformers	Full observation (video)	Visual-generative	Training (env. generation)
WMA Web Agent (Chae et al., 2025)	Web	State delta (web transition)	LLM-based	Planning (policy selection)
WebWM (Feng et al., 2025a)	Web	State delta (web dynamics)	Hybrid/modular	Training (env. generation)
WebWorld (Xiao et al., 2026)	Web	Full observation (web page)	LLM-based	Training, planning
DynaWeb (Ding et al., 2026)	Web	Full observation (web page)	LLM-based	Training (model-based RL)
WebEvolver (Fang et al., 2025)	Web	State delta (web state)	LLM-based	Training (co-evolution), planning
VAGEN (Wang et al., 2025a)	Web / GUI	Latent transition (belief)	VLM/RL-based	Training (world-model reasoning)
R-WoM (Mei et al., 2026)	GUI / desktop	State delta (UI change)	Hybrid/modular	Planning, verification
ViMo (Luo et al., 2025a)	GUI / mobile	Full observation (GUI image)	Visual-generative	Planning (action evaluation)
MobileDreamer (Cao et al., 2026)	GUI / mobile	Full observation (GUI sketch)	LLM-based	Planning (action evaluation)
Code2World (Zheng et al., 2026)	GUI / mobile, web	Full observation (renderable code)	Code-based	Planning (simulation)
Computer-Using WM (Guan et al., 2026)	GUI / desktop	State delta (UI change)	LLM-based	Planning (action search)
SafePred (Chen et al., 2026)	GUI / desktop	Auxiliary outcome (risk)	LLM-based	Verification (guardrail)
AgentWM (Wang et al., 2026)	Tool-use	Full observation (tool state)	Code-based	Training (env. generation)
GTM (Ren et al., 2025)	Tool-use	Full observation (tool output)	LLM-based	Training (tool simulation)
ToolRM (Agarwal et al., 2025)	Tool-use	Auxiliary outcome (tool reward)	LLM-based	Verification
WorldCoder (Tang et al., 2024)	Code	State delta (program dynamics)	Code-based	Planning
Code World Models (Dainese et al., 2024)	Code	State delta (program dynamics)	Code-based	Planning, verification
CWM (FAIR CodeGen team et al., 2025)	Code	State delta (execution trace)	LLM-based	Verification
SWE-World (Sun et al., 2026b)	Code / SWE	Auxiliary outcome (execution/test feedback)	LLM-based	Training, verification

3.3 Agent-Facing Role ($\mathcal{U}$)

The third dimension, $\mathcal{U}$, characterizes **agent-facing role**, that is, **how it is used** inside the agent loop. This is the dimension that most clearly distinguishes an agent-centric analysis from a purely model-centric one. Two models with similar predictive capabilities may have very different practical significance depending on what role their predictions play for the agent.

Decision-time planning A world model may be queried to simulate candidate actions, compare hypothetical futures, or support multi-step search before execution. In this role, it converts action selection from a purely reactive process into a deliberative one.

Training-time interaction A world model may also be used during training to generate synthetic trajectories, provide a surrogate environment, expand task diversity, or support self-improvement. Here it serves as a data engine or training substrate rather than purely as an inference-time planning tool.

Predictive verification In digital settings, a world model may further be used to examine candidate plans, assumptions, or actions before execution. By projecting their likely consequences, the model can support feasibility checking, consistency testing, counterfactual comparison, and safety-aware validation. This role is especially important when errors arise from hidden preconditions, delayed constraints, or irreversible state changes that are not immediately visible from the current observation (Yao et al., 2023b; Hao et al., 2023).

These roles are analytically distinct, even when they rely on related predictive mechanisms. For example, an LLM-based transition model may support one-step lookahead for action selection (Gu et al., 2025) (planning), generate synthetic browsing trajectories for policy training (Xiao et al., 2026) (training-time interaction), or predict whether an action risks irreversible consequences (Chen et al., 2026) (verification). The distinction therefore lies not only in what the model predicts, but in what function those predictions serve for the agent.

3.4 Connecting the Three Dimensions

The three dimensions of $\mathcal{W} = (\mathcal{X}, \mathcal{L}, \mathcal{U})$ are conceptually distinct but tightly coupled. A digital-agent world model is valuable not simply because it can predict what comes next, but because it exposes the right action-relevant future, in the right form, built through the right operational pathway, for the right agent-facing role. Table 2 instantiates this design space by mapping representative methods to their approximate $(\mathcal{X}, \mathcal{L}, \mathcal{U})$ coordinates across all four digital domains. In practice, this means that usefulness depends on alignment across all three dimensions. A simple semantic predictor may be more useful than a highly expressive generative model if it better matches the agent's decision requirements. Conversely, a strong predictive

target may remain practically ineffective unless supported by suitable training data, executable grounding, or an appropriate usage strategy. This perspective organizes the remainder of this survey.

4 Building World Models for Digital Agents

This section instantiates the construction side of the design space $\mathcal{W} = (\mathcal{X}, \mathcal{L}, \mathcal{U})$ by focusing on $\mathcal{X}$ (what is modeled) and $\mathcal{L}$ (how it is built). Action effects in digital environments can propagate through hidden dependencies, asynchronous updates, and remote services (Section 2). Building world models is therefore best understood as a problem of selective predictive fidelity: the model should preserve action-critical consequences while remaining abstract enough to be efficient, robust, and verifiable over long horizons.

4.1 What Is Modeled: State Abstraction and Prediction Targets

The first design decision concerns the representational form of future consequences. Digital observations such as screenshots, DOM trees, accessibility graphs, repository files, execution traces, or tool outputs are often high-dimensional and only partially reveal the underlying software state. Choosing what to predict therefore means choosing which invariances to preserve across transitions and which details to abstract away.

Existing work can be organized into four broad families: *state-delta models*, *full-observation models*, *latent-transition models*, and *auxiliary outcome predictors*. These families differ not only in representational granularity, but also along four sharper comparison axes: *information coverage* (how much future structure is retained), *semantic alignment* (how directly the prediction corresponds to action-relevant change), *rollout efficiency* (how costly multi-step composition is), and *verifiability* (how easily the predicted future can be checked against execution or constraints). Read this way, the four families are not merely descriptive categories. They are competing ways of deciding what future the agent is allowed to reason over.

4.1.1 State deltas

Many digital actions induce sparse or semantically structured changes: clicking a button, editing a field, calling a tool, or modifying a file typically affects only a small subset of the overall environment state. State-delta models exploit this sparsity by predicting the change an action induces on the relevant state variables rather than reconstructing the entire next observation.

In web environments, predicted deltas often take the form of structured or natural-language descriptions of inserted, removed, or updated interface elements (Chae et al., 2025; Gu et al., 2025; Feng et al., 2025a). Similarly, GUI world models may predict which widgets change state, which dialogs appear, or which errors are triggered (Guan et al., 2026; Mei et al., 2026). Related ideas arise in tool-use and code environments (Tang et al., 2024; Dainese et al., 2024). Planning-oriented game world models also emphasize compact action-conditioned change, although usually in latent or value-equivalent form rather than as explicit semantic deltas (Schrittwieser et al., 2020). The main strength of delta prediction is semantic alignment: the agent often cares far more that an action opens a confirmation dialog or mutates a key variable than what every unaffected region of the screen looks like, making delta prediction a highly decision-efficient predictive interface when action effects are sparse and semantically local.

Its limitation is that alignment is purchased through selectivity. When layout dependencies, hidden backend state, global constraints, cross-module interactions, or long-range workflow effects determine what matters next, localized deltas may omit exactly the structure that later decisions depend on. State-delta models therefore perform best when action consequences are sparse and semantically local, but become fragile when the true future depends on global context or latent state not captured by the chosen delta representation.

4.1.2 Full observations

At the opposite extreme, full-observation models predict the complete next interface state, such as a screenshot, rendered page, accessibility tree, future HTML document, or other holistic interface representation. This choice imposes fewer assumptions on which aspects of the future will matter, and is therefore attractive when the relevant abstraction is not known in advance.

Visual world models for GUI agents generate next-screen observations conditioned on action histories, supporting imagined rollouts, synthetic trajectory construction, and scalable simulator building (Luo et al., 2025a; Xiang et al., 2025; Cao et al., 2026). Related web-oriented systems produce future page representations for policy learning and model-based interaction (Xiao et al., 2026; Ding et al., 2026; Wang et al., 2026). An important intermediate variant instead predicts renderable intermediate representations such as HTML/CSS or structured UI programs, which can then be rendered and verified programmatically (Zheng et al., 2026; Koh et al., 2026).

The central trade-off is between coverage and decision density. Full-observation models preserve much more future information than delta models, but a large fraction of that information may be irrelevant to action selection. As a result, they are computationally heavier, more vulnerable to compounding rollout error, and often judged by perceptual similarity measures that do not faithfully reflect functional correctness. A visually plausible future can still be decision-useless if it misses the disabled button, altered form field, hidden workflow transition, or external-state mutation that actually determines the next best action. Thus, full-observation prediction is best viewed as a high-coverage but low-selectivity interface: it is valuable when relevant consequences are diffuse or hard to specify, but costly when only a small subset of the future matters.

4.1.3 Latent transitions

Latent-transition models represent dynamics in a learned hidden space rather than directly predicting observations. An encoder maps the current observation into a compact latent state, a transition model predicts future latent states conditioned on actions, and auxiliary heads may decode observations or estimate reward-, value-, or continuation-related signals.

This paradigm originates in RSSM-style architectures and Dreamer (Hafner et al., 2020; 2025), planning-oriented latent dynamics such as MuZero (Schrittwieser et al., 2020), and transformer-based variants over tokenized latent trajectories (Micheli et al., 2023; Zhang et al., 2023). In digital-agent settings, latent representations are increasingly adopted as internal components: for example, VAGEN uses latent world-model-guided belief states to improve multi-turn reasoning in web and GUI agents (Wang et al., 2025a).

Their main advantage is rollout efficiency. By avoiding explicit reconstruction at every step, they provide an efficient substrate for long-horizon imagination, search, and simulation-heavy learning. Their weakness is semantic opacity: in digital-agent regimes, latent states are often difficult to align with explicit interface semantics, typed tool constraints, or executability requirements. Purely latent world models therefore remain more dominant in game-like settings. In web, GUI, or tool-use agents, they are often most effective not as the visible predictive interface, but as the internal rollout substrate beneath more interpretable semantic, structural, or executable prediction heads.

4.1.4 Auxiliary outcome prediction

Many digital world models predict not only future states, but also outcome-level signals such as execution success, task progress, constraint violation, or future risk. From an agent-centric perspective these are best understood as decision-oriented predictive targets. In code environments, predicted edits can be evaluated against tests or runtime behavior (Tang et al., 2024; Dainese et al., 2024); in software engineering, models may predict whether an edit preserves repository integrity (Sun et al., 2026b); in safety-sensitive GUI settings, predictive guardrails estimate whether an action may trigger irreversible or policy-violating futures (Chen et al., 2026); and in game-centered planning, auxiliary predictions include value or branch-utility estimates that guide search (Schrittwieser et al., 2020; Bruce et al., 2024).

The defining strength of this family is decision directness. A model that predicts whether a form submission will fail, whether a command will corrupt the environment, or whether a candidate branch is promising may be more useful than one that reconstructs the full next state with higher fidelity but says nothing about feasibility. The limitation is equally important: auxiliary targets depend heavily on how the downstream decision problem is defined. If the chosen outcome variable is too narrow, the model may become myopic, overfitting to a proxy while missing broader environment structure the agent later needs.

Takeaway The four target families answer the same question from different angles: state deltas maximize semantic selectivity, full observations maximize coverage, latent transitions maximize rollout efficiency, and auxiliary predictors maximize decision directness. None dominates universally. The key design principle is which target preserves the minimal future structure required for the intended usage regime in $\mathcal{U}$. The empirical trend is toward multi-granular predictive interfaces that combine several targets, because in digital environments, the best predictive target is rarely the most detailed one but the one that preserves the distinctions the agent cannot afford to get wrong.

4.2 How It Is Modeled: Architectural Paradigms

Once the predictive target is chosen, the next question is how to realize it computationally. Digital-agent transitions are intrinsically heterogeneous (symbolic updates, rendered interface evolution, typed tool effects, executable programs), so architectural paradigms are best understood as different ways of instantiating distinct kinds of predictive interfaces.

We evaluate paradigms along four criteria: *semantic faithfulness*, *rollout cost*, *operational verifiability*, and *modality fit*, and organize existing work into five paradigms: *LLM-based transition models*, *visual generative models*, *latent dynamics architectures*, *code-based world models*, and *hybrid or modular systems*.

4.2.1 LLM-based transition modeling

When environment state can be serialized as text or semi-symbolic structure, transition prediction can be formulated as conditional language generation. An LLM consumes a representation of the current state together with a candidate action and produces a predicted next state, semantic delta, or future consequence summary (Gu et al., 2025; Chae et al., 2025; Feng et al., 2025a).

This paradigm is especially natural in digital-agent settings because many software environments are already text-compatible: HTML, DOM trees, accessibility structures, tool schemas, code diffs, and workflow summaries can all be serialized into tokens. LLMs can therefore leverage strong pretrained priors about websites, interfaces, APIs, and software workflows, making them effective semantic simulators for web navigation, computer-use reasoning, and multi-step tool interaction (Gu et al., 2025; Shen et al., 2026; Xiao et al., 2026; Guan et al., 2026). For tool-use agents, LLM-based transition models are particularly attractive because API schemas, function signatures, and structured return values are natively expressible as text. The model can predict how an API call mutates external state, whether argument combinations satisfy cross-parameter constraints, and how one tool invocation's side effects alter the preconditions of subsequent calls (Wang et al., 2026; Yao et al., 2025; Patil et al., 2025). This makes LLM-based modeling a natural substrate for tool-state forecasting, predicting the persistent state changes induced by tool sequences across services, databases, and external systems (Gupta et al., 2026; Barres et al., 2025).

Their central strength is semantic flexibility: effectiveness when the relevant consequence is symbolic or workflow-like. Their main bottleneck is that everything depends on the chosen serialization. Their dominant failure mode is serialization-induced omission: the model reasons fluently over a representation that has already discarded the state distinctions that matter. They are strongest when semantic consequence matters more than pixel-level evolution.

4.2.2 Visual generative models

When agents act directly over screenshots or rendered interfaces, visual generative models provide a natural mechanism for future prediction. These models generate future interface observations conditioned on interaction histories, supporting imagined rollouts, off-device simulation, and synthetic experience generation (Luo et al., 2025a; Cao et al., 2026; Xiang et al., 2025). Related ideas also appear in game-oriented world models that learn controllable visual dynamics for planning and simulation (Bruce et al., 2024).

Their main advantage is high observational coverage. They preserve broad visual context, retain layout-level consequences, and naturally match settings in which action is grounded directly in pixels or rendered UI.

However, their weakness is that perceptual realism is only an indirect proxy for functional correctness. A world model for GUI interaction is useful only insofar as the generated future preserves action affordances, layout logic, interface consistency, and semantically relevant state changes.

For this reason, recent work increasingly augments pure visual generation with structure-aware conditioning, such as UI hierarchies, DOM representations, or renderable intermediate code (Zheng et al., 2026; Koh et al., 2026). Compared with LLM-based models, visual generators usually preserve more future information but provide a weaker built-in bias toward action-relevant abstraction. Their dominant failure mode is surface correctness without operational correctness: a screen can look plausible while being unusable for planning.

4.2.3 Latent dynamics architectures

Latent dynamics architectures provide a different trade-off: they compress environment evolution into hidden states that can be rolled out efficiently over many steps without explicitly generating full observations at each step. This family includes RSSM-style recurrent models (Hafner et al., 2020; 2025), planning-focused latent models such as MuZero (Schrittwieser et al., 2020), and transformer-based latent predictors (Micheli et al., 2023; Zhang et al., 2023).

Their key strength is computational leverage: they are often the most efficient option for deep rollout, search, and simulation-heavy optimization. Their main weakness is that useful digital-agent behavior often depends on distinctions hard to leave implicit: tool validity, typed argument structure, interface affordances, or executable constraints. Their dominant failure mode is thus efficient but ungrounded abstraction. For this reason, digital-agent systems increasingly use latent rollout as an internal computational substrate paired with more interpretable semantic or executable output heads (Guan et al., 2026; Zheng et al., 2026).

4.2.4 Code-based world models

Digital environments are unusual in that their transition dynamics can sometimes be represented directly as executable programs. Instead of storing predictive structure only in neural parameters, the model generates code that simulates transitions, renders interfaces, checks constraints, or evaluates action outcomes (Tang et al., 2024; Zheng et al., 2026; Dainese et al., 2024). Their appeal is threefold: predicted dynamics are explicit and inspectable, predictions can be verified through execution or testing, and the approach naturally aligns with domains already governed by symbolic rules or executable semantics.

This paradigm is especially relevant to code and software-engineering domains, where the environment is code and the transition dynamics are governed by compilation, testing, and execution. WorldCoder synthesizes executable programs that model environment dynamics and uses them as substrates for MCTS-based planning (Tang et al., 2024). SWE-World constructs surrogate software-engineering environments by predicting whether proposed code edits preserve repository integrity, effectively modeling the code-to-test-outcome transition as a world model for agent training (Sun et al., 2026b). Code2World generates renderable HTML/CSS artifacts that simulate GUI transitions, bridging code-based prediction with visual verification (Zheng et al., 2026). More broadly, code-based world models are natural whenever the predictive target can be expressed as a program whose execution is the predicted future: the model's output is not a description of what will happen, but a program that makes it happen, enabling direct verification through execution rather than indirect comparison through similarity metrics.

Their core strength is therefore verifiable consequence modeling. Compared with latent or purely textual predictors, they provide one of the clearest pathways from predicted future to external check. The main challenge concerns abstraction level and control. Generated programs may simulate low-level transitions, express higher-level workflow logic, or instantiate partially executable intermediate representations. Their dominant failure mode is brittle explicitness: the generated structure is inspectable and testable, but may be hard to scale, generalize, or adapt when the underlying environment is noisy or weakly formalized.

4.2.5 Hybrid and modular systems

Because digital-agent transitions are heterogeneous, many recent systems combine multiple predictive components within one agent loop, such as an LLM-based semantic transition model, a visual renderer, a retrieval

module, and an executable verifier (Shen et al., 2026; Xiao et al., 2026; Guan et al., 2026; Zheng et al., 2026). This reflects a deeper property of digital environments: different aspects of the same transition may be best captured by different representational mechanisms, so hybrid architectures often achieve better trade-offs among expressiveness, efficiency, and verifiability than any single mechanism.

Their key strength is complementarity. Their key challenge is coordination: multiple predictive modules must remain aligned, calibrated, and cost-effective. The dominant failure mode is not necessarily bad local prediction, but inconsistency across modules, for example when a semantic predictor, renderer, and verifier disagree about what future is actually implied.

Takeaway LLM-based models are strongest in semantic flexibility, visual models in observational coverage, latent models in rollout efficiency, code-based models in verifiability, and hybrid systems in cross-mechanism complementarity. The main architectural question is not "which backbone is best?" but which failure mode is most acceptable for the target environment and usage regime? The strongest empirical trajectory is toward modular predictive infrastructures that separate semantic abstraction, efficient rollout, and external verification rather than forcing a single model family to handle all predictive tasks equally well.

4.3 How It Is Learned: Data Construction and Training

Choosing a predictive target and architecture is only part of the problem. In digital-agent settings, world-model quality depends just as critically on how action-conditioned supervision is collected, how coverage is expanded beyond narrow demonstrations, and what training objectives most effectively align local prediction with downstream agent behavior.

Digital environments induce highly combinatorial interaction spaces, and observed trajectories typically cover only a thin slice. The central challenge is action-conditioned coverage construction: collecting the right future consequences under the right intervention structure. Whether supervision comes from offline logs, online exploration, or synthetic generation, the bottlenecks recur: *coverage skew*, *branch sparsity*, *failure underexposure*, *objective mismatch*, and *distribution drift*.

4.3.1 Data sources

Existing work relies on three primary sources of supervision. The first is offline interaction logs, including human demonstrations or historical agent traces over websites, mobile apps, desktops, operating systems, code tasks, and game trajectories (Deng et al., 2023; Rawles et al., 2023; Jimenez et al., 2024; Bellemare et al., 2013). These logs provide realistic state-action-next-state evidence, but they often overrepresent successful workflows and underrepresent failure, ambiguity, and recovery.

The second is online interaction, where agents actively collect fresh transitions from live or executable environments (Ding et al., 2026; Fang et al., 2025; Rawles et al., 2025). Online collection improves coverage under the current policy and adapts to changing interfaces, but it is slow, expensive, and in real software systems may introduce operational risk.

The third is synthetic generation, where trajectories, tasks, or even entire training environments are generated rather than directly observed (Xiao et al., 2026; Wang et al., 2025b; 2026; Bruce et al., 2024). Synthetic data can scale quickly and target underrepresented regions of the interaction space, but risks amplifying simulator bias when generated futures diverge from executable ground truth. These three sources are not interchangeable: offline data offers realism, online data offers adaptation, and synthetic data offers coverage expansion. Strong training pipelines increasingly combine all three because no single source simultaneously provides realistic execution semantics, counterfactual coverage, failure diversity, and adaptation under drift.

4.3.2 Coverage and diversity

World models must not be trained only on successful demonstrations: a model that only observes successful trajectories may predict well near demonstrations but fail when the agent deviates, encounters ambiguity, or must recover from an intermediate mistake (Ding et al., 2026; Feng et al., 2026; Fang et al., 2025). This issue is especially severe in digital environments because many consequential transitions are sparse: a single disabled button, hidden precondition, failed API call, authentication state, or irreversible operation may determine task success while appearing rarely in passive logs. Thus, coverage should be understood not merely as the number of trajectories, but as the diversity of action-conditioned branches that expose how the environment changes under meaningful interventions.

A useful coverage distribution should include at least three types of variation. The first is task and state diversity: the model should observe the same action under different pages, files, tool states, repositories, or game contexts. The second is branch diversity: the data should include alternative actions from the same or similar states, so that the model can distinguish near-miss decisions from successful ones. The third is failure and recovery diversity: the model should observe invalid actions, constraint violations, partial progress, and recovery attempts, rather than only completed solutions. Recent work broadens such coverage through online self-improvement, agent–environment co-evolution, and large-scale simulator construction (Mei et al., 2026; Feng et al., 2026; Xiao et al., 2026). These mechanisms are valuable because they expose the model to counterfactual and off-policy regions that are unlikely to appear in static demonstrations.

The key constraint is therefore not raw sample count but support over decision-critical transitions. A dataset with millions of nominal interactions may be less useful than a smaller one that covers irreversible actions, rare failures, hidden preconditions, and recovery paths. For digital-agent world models, coverage is most valuable when it is intervention-aware: it should reveal not only what usually happens, but also what would happen if the agent chose a different action at a consequential branch point.

4.3.3 Training objectives

Training objectives depend on the predictive interface being learned. LLM-based world models optimize next-token prediction over serialized transitions, deltas, or future summaries (Chae et al., 2025; Xiao et al., 2026; Feng et al., 2025a). Visual world models use reconstruction or generative objectives over screenshots, render targets, or action-conditioned frames (Luo et al., 2025a; Cao et al., 2026; Bruce et al., 2024). Code-based world models supplement token-level modeling with execution-aware feedback, so that predicted programs are judged by what they do, not only by what they look like (Tang et al., 2024; Dainese et al., 2024; Zheng et al., 2026). Latent models combine transition learning with reward, value, or continuation prediction to align hidden dynamics with planning utility (Schrittwieser et al., 2020; Hafner et al., 2025).

More broadly, these objectives can be organized as optimizing four progressively stronger notions of correctness: surface fidelity, semantic validity, executability, and decision utility. This layered view is particularly important in digital settings because the objective mismatch is unusually explicit, and because digital environments frequently expose native functional feedback: code can be directly executed, HTML can be rendered, tool outputs can be checked against schemas, and risky futures can be labeled under policy constraints (Chen et al., 2026). Objective design can therefore increasingly target decision-relevant correctness rather than surface-level fidelity alone.

4.3.4 Two-stage learning and decision-oriented refinement

A common pattern is to first learn a broad predictive model from offline, online, or synthetic interaction data, and then refine it with signals closer to downstream utility. The first stage provides general transition knowledge: how pages change after clicks, how tools mutate external state, how code execution responds to edits, or how latent game states evolve. However, a model trained only for average predictive fit may allocate capacity to frequent but low-value transitions, while under-modeling rare events that dominate decision quality. This motivates a second stage in which learning is reweighted toward the branches, errors, and feedback signals that matter for agent behavior.

Decision-oriented refinement can take several forms. Search-guided selection uses predicted futures to choose or filter trajectories that better support downstream action selection (Gao et al., 2025). Imagined-rollout policy improvement trains agents on model-generated interaction branches, thereby turning the world model into a surrogate source of experience (Ding et al., 2026). Render-aware or execution-aware reinforcement learning refines predictions according to whether generated artifacts actually render, execute, or satisfy task constraints (Zheng et al., 2026). Continual self-improvement further updates the model as the agent discovers new states, failures, and recovery strategies during interaction (Feng et al., 2026; Fang et al., 2025). Across these variants, the goal is not simply to increase average next-state accuracy, but to build a decision-support module whose errors are smallest where they affect action choice, verification, or safety (Gu et al., 2025; Chae et al., 2025; Chen et al., 2026).

Refinement exposes a trade-off. If refinement is too tightly coupled to the current policy, the world model may become accurate only around the agent's existing behavior and fail under exploration. If refinement optimizes only rollout plausibility, the model may generate coherent but non-executable futures. Effective pipelines therefore combine broad pretraining with functional feedback, hard negative branches, and continual correction from real or executable environments. Two-stage learning is not merely a training recipe. It is a way to align predictive modeling with the downstream role the world model is expected to serve.

Takeaway Learning digital world models is fundamentally a problem of decision-aligned coverage construction. The decisive resource is not raw data volume, but coverage over consequential branches: the transitions that determine downstream utility are often the least likely to appear in standard data. This is why the strongest pipelines increasingly combine offline realism, online adaptation, synthetic branch expansion, and functional feedback rather than relying on passive next-state prediction alone. Better learning comes less from fitting averages and more from supervising exceptions, alternatives, and failures deliberately.

4.4 Construction-Time Challenges

Despite rapid progress, several recurring difficulties cut across predictive targets, architectural paradigms, and learning strategies. These are best understood as construction-time bottlenecks: they arise while specifying $\mathcal{X}$ and realizing $\mathcal{L}$, before downstream usage in $\mathcal{U}$ can succeed.

Partial observability and hidden software state Digital agents typically observe only interface-level manifestations of deeper system state. Screenshots, DOM trees, tool responses, and game frames reveal only part of what determines the true action consequence. Authentication status, hidden form values, server-side state, and latent game variables may all affect transitions while remaining invisible (Deng et al., 2023; Zhou et al., 2024b; Schrittwieser et al., 2020). This makes predictive abstraction necessary but also dangerous: the model must infer what matters without direct access to the full environment state. Effective systems therefore often incorporate longer interaction histories, retrieval support, structured memory, or calibrated uncertainty estimates to compensate.

Long-horizon error accumulation Digital tasks frequently require multi-step foresight, yet small local mistakes can compound across imagined rollouts until the simulated future becomes unusable. This challenge is especially acute for full-observation models and unconstrained language-based rollouts, where slight drift at one step can derail all later predictions (Mei et al., 2026; Qian et al., 2026). The problem is not unique to digital settings, but digital workflows often make it especially costly because one early modeling mistake may invalidate an entire chain of later actions. Current mitigation strategies include shallow lookahead with frequent re-grounding, retrieval support, uncertainty-aware pruning, and co-adaptation between policy and world model over states that actually arise in practice (Mei et al., 2026; Feng et al., 2026; Fang et al., 2025).

Heterogeneous states, actions, and constraints Digital agents operate over unusually heterogeneous modalities: states may contain screenshots, DOM trees, tool outputs, or repository files, while actions may

include clicks, typing, API calls, code edits, or higher-level workflow operators (Guan et al., 2026; Gupta et al., 2026; Wang et al., 2025c). Serializing everything into text offers one unifying route but often discards useful structure. Building separate predictors for each modality improves fidelity but increases system complexity. Designing world models that natively handle such heterogeneity while preserving compositionality remains a major challenge for generalist digital agents.

When world models hurt performance Not all world models help. Rare but consequential transitions are systematically underrepresented in logged data, and this problem is intensified by continual environment drift as websites evolve, APIs change schemas, and software ecosystems expand (Xiao et al., 2026; Wang et al., 2025b; Mei et al., 2026). More fundamentally, the objective mismatch identified by massoud Farahmand (2018) and Lambert et al. (2020) implies that optimizing predictive fidelity can actively mislead when the learned model captures surface regularities but misses the sparse functional distinctions that determine task success. Policies may exploit model inaccuracies, achieving high reward under the model while performing poorly in the real environment, and compounding rollout error can cause model-based planning to underperform simpler reactive policies (Qian et al., 2026). More broadly, recent evidence suggests that current digital agents often struggle to effectively leverage explicit world models at all, raising the question of whether the construction cost is justified when strong foundation models may already capture sufficient transition knowledge (Qian et al., 2026). These negative results bound the conditions under which explicit world modeling is beneficial and underscore the importance of cost-aware design.

Computational cost and practical deployability The inference overhead of world models is rarely reported but is a critical practical consideration. LLM-based transition models require a full forward pass for each candidate action, which at decision time can multiply latency by the branching factor of the search. Visual generative models face even higher per-step costs due to image generation, making deep rollout expensive. Latent dynamics models are the most efficient per rollout step, but incur significant upfront training cost to learn useful representations. Code-based models add execution overhead for rendering or running generated programs. In practice, these costs often determine whether a world model can be deployed inside an agent loop at all, yet most current work reports only downstream task performance without disaggregating the computational budget consumed by world-model inference versus direct environment interaction. Systematic cost-benefit analysis remains an important gap.

A unifying view These challenges all reflect one underlying difficulty: digital world models must preserve the right hidden distinctions under sparse, drifting, and heterogeneous supervision. Partial observability hides those distinctions, long-horizon rollout magnifies mistakes about them, surface-level objectives may reward visually plausible predictions that miss the functional consequence that matters, heterogeneity makes them hard to represent uniformly, and distribution shift ensures they keep changing. The most promising systems are therefore those that combine compact but semantically meaningful predictive targets, architectures matched to environment modality, and learning pipelines optimized for functional utility rather than reconstruction quality alone.

5 How to Use World Models in Digital Agents

In classical model-based reinforcement learning, world models mainly improve control and sample efficiency through imagined rollouts (Ha & Schmidhuber, 2018; Hafner et al., 2020; Schrittwieser et al., 2020; Hafner et al., 2025). For digital agents, $\mathcal{U}$ encompasses a broader set of roles (Chae et al., 2025; Gu et al., 2025; Guan et al., 2026; Feng et al., 2025a; Ding et al., 2026). As illustrated in Figure 2, existing usage patterns fall into three complementary paradigms: **decision-time planning** (querying models at inference to evaluate candidate actions), **training-time interaction** (generating synthetic experience or surrogate environments), and **predictive verification** (assessing feasibility, consistency, or validity of candidate plans). These paradigms are analytically distinct but increasingly combined in practice.

5.1 Decision-Time Planning

Decision-time planning uses world models to evaluate hypothetical futures before committing to an action (Figure 2, left). This idea has deep roots in model-based planning (Silver et al., 2016; Schrittwieser et al., 2020; Vinyals et al., 2019), but is especially valuable for digital agents because many mistakes are both operationally costly and predictable (Chae et al., 2025; Gu et al., 2025; Guan et al., 2026; Mei et al., 2026; Côté et al., 2018; Wang et al., 2022; 2025c).

Formally, let $\tau = (s_t, a_t, s_{t+1}, a_{t+1}, \ldots, s_{t+H})$ denote an imagined rollout of horizon H, generated by repeatedly applying a learned world model $p_\theta(s_{k+1} \mid s_k, a_k)$. Decision-time planning selects the current action by maximizing expected utility over such imagined rollouts:

$$a_t^* = \arg\max_{a_t} \; \mathbb{E}_{\tau \sim p_\theta}\big[R(\tau)\big],$$

where $R(\tau)$ denotes the cumulative task utility or return along the rollout. In practice, exact optimization is intractable in high-dimensional digital environments, and existing methods approximate this objective under limited inference-time compute. We group them into three representative families.

5.1.1 Short-horizon lookahead

The simplest pattern is short-horizon lookahead: the world model predicts the immediate consequence of each candidate action and ranks them by predicted one-step outcome. This suffices when the next state reveals whether an action is on-task, safe, or reversible. In web, GUI, and computer-use settings, next-state prediction can expose changes in DOM structure, screen layout, tool status, or program state, filtering obviously poor actions before execution (Chae et al., 2025; Luo et al., 2025a; Li et al., 2025a; Cao et al., 2026; Guan et al., 2026; Côté et al., 2018; Wang et al., 2022). Short-horizon lookahead is the most practical entry point because it demands limited rollout fidelity while still providing decision-relevant foresight.

5.1.2 Search over imagined trajectories

A richer pattern couples the world model with explicit search over multi-step imagined trajectories, rolling out learned dynamics over a horizon H and selecting actions by optimizing cumulative predicted return via beam search, MCTS, or related procedures (Tang et al., 2024; Dainese et al., 2024; Gao et al., 2025; Deng et al., 2025; Gu et al., 2025; Silver et al., 2016; Schrittwieser et al., 2020; Vinyals et al., 2019). This is natural in digital environments because candidate futures are structured around interface operations, tool chains, or code edits rather than low-level motor trajectories. Compared with one-step lookahead, search-based planning is better suited to tasks with delayed consequences or strong cross-step dependencies, but places stronger demands on world-model fidelity and uncertainty calibration.

5.1.3 Test-time scaling

A third pattern improves decision reliability by allocating more inference-time compute to world-model querying rather than to parameter updates. Concretely, the agent samples N imagined futures for each candidate action and selects the action whose sampled rollouts yield the highest aggregate predicted return, where the aggregation may be a mean, a majority vote, a top-K filter, or another operator over the individual rollout utilities. This is particularly useful in digital settings because real failures are expensive and hard to recover from. Extra compute on imagined futures is often far cheaper (Gu et al., 2025; Gao et al., 2025; Deng et al., 2025; Guan et al., 2026; Mei et al., 2026). It connects to the broader trend of increasing inference-time deliberation, but with a specific focus: the compute is spent on evaluating consequences under modeled environment dynamics. We note that several inference-time deliberation methods (Yao et al., 2023b; Hao et al., 2023; Yao et al., 2023a; Long, 2023; Shinn et al., 2023; Zhou et al., 2024a) rely on the language model's internal capacity to simulate state transitions, constituting an implicit form of world modeling under the framework of Section 3.

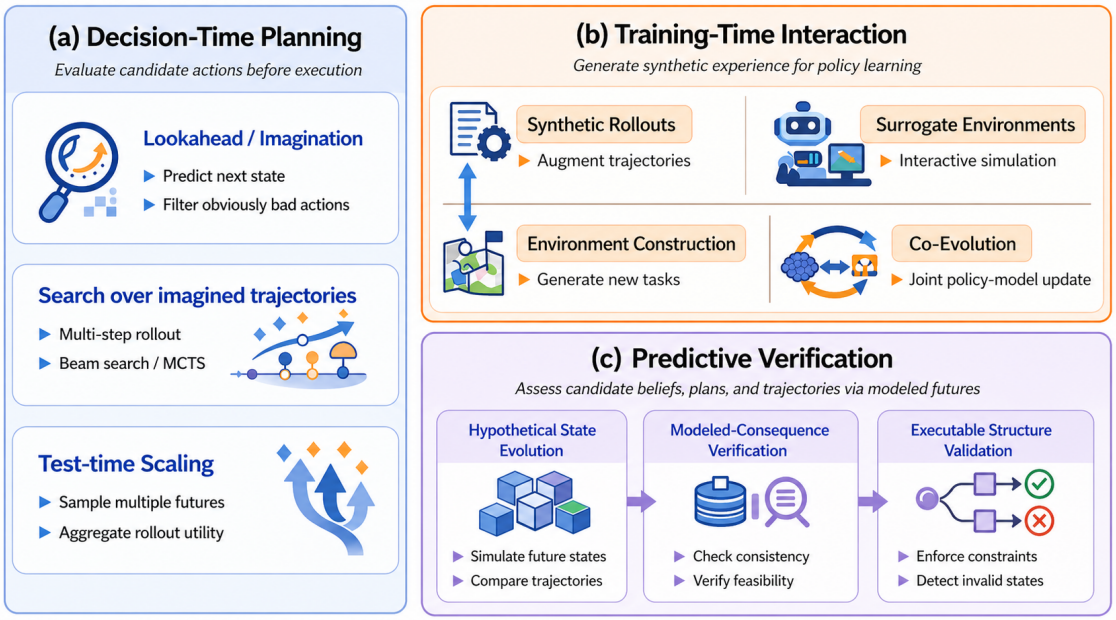

Figure 2: Three usage patterns of world models inside the digital-agent loop. Decision-time planning uses modeled futures to evaluate candidate actions before execution; training-time interaction uses synthetic rollouts or surrogate environments to improve agents during learning; and predictive verification uses modeled consequences to check feasibility, consistency, and risk before commitment.

Takeaway Taken together, short-horizon lookahead, search over imagined trajectories, and test-time scaling can all be viewed as different approximations to the same underlying planning objective. They reflect a common compute–accuracy trade-off in how world models are queried at inference time. Local tasks often benefit from cheap one-step foresight, whereas long-horizon, high-stakes, or highly compositional tasks may justify deeper search and broader rollout aggregation.

5.2 Training-Time Interaction

World models also reduce reliance on expensive real interaction by generating synthetic experience for training, following the classic imagination-based view of model-based learning (Hafner et al., 2019; 2020; 2025; Micheli et al., 2023; Zhang et al., 2023). This role is especially important for digital agents because collecting real interaction data is often slow, risky, or operationally constrained (Zhou et al., 2024b; Xie et al., 2024; Yao et al., 2025; Sun et al., 2026b). As illustrated in Figure 2 (top right), training-time usage forms a spectrum from *open-loop* synthetic trajectory augmentation to *closed-loop* interaction with learned surrogate environments, then to *environment-level construction*, and ultimately to the *co-evolution* of world models and policies, differing in how tightly the policy is coupled to modeled dynamics during learning.

5.2.1 Open-loop synthetic trajectory augmentation

In this setting, the world model generates imagined state transitions conditioned on observed states and candidate actions, augmenting logged data without determining the policy's online exploration. This is conceptually aligned with imagination-based policy learning (Hafner et al., 2020; 2025; Micheli et al., 2023), but digital-agent work emphasizes traces that remain semantically plausible under interface constraints. Web-oriented methods produce large-scale synthetic browsing experience (Fang et al., 2025; Xiao et al., 2026; Ding et al., 2026; Wang et al., 2026; Feng et al., 2025a). Similar approaches appear in mobile and GUI settings (Li et al., 2025a; Xiang et al., 2025; Cao et al., 2026) and in text-interaction environments (Côté et al.,

2018; Wang et al., 2022; 2025c). For digital agents, synthetic trajectory generation systematically enlarges the reachable task manifold while preserving the operational structure of the underlying environment. Its effectiveness depends on predictive fidelity: if generated transitions deviate substantially from executable dynamics, synthetic data may introduce bias rather than useful diversity.

5.2.2 Closed-loop interaction with surrogate environments

A stronger form treats the world model as a surrogate environment in which the policy interacts online, step by step (Ha & Schmidhuber, 2018; Hafner et al., 2020; 2025). This is especially attractive in digital settings because many transitions are discrete and at least partially verifiable. Recent work builds scalable computer-use, software-engineering, and workflow-oriented surrogates (Sun et al., 2026b; Guan et al., 2026; Gupta et al., 2026; Wang et al., 2026), complementing ecosystem-level benchmarks that motivate realistic training grounds (Zhou et al., 2024b; Rawles et al., 2025; Xie et al., 2024; Bonatti et al., 2025; Yao et al., 2025; Kong et al., 2025b). Compared with open-loop augmentation, closed-loop interaction allows the agent to discover recovery behavior and compounding errors induced by its own policy. Its main challenge is that once the agent begins to exploit systematic modeling errors, simulated interaction may drift toward unrealistic but apparently high-value trajectories.

5.2.3 Generative environment construction

World models can also contribute to environment construction: shaping the distribution of tasks, interfaces, and interaction structures the agent encounters during training. This is natural in digital domains because interfaces and coding tasks can often be represented as executable artifacts. Code-based or renderable world construction has emerged in GUI and coding settings (Tang et al., 2024; Zheng et al., 2026; Koh et al., 2026; Sun et al., 2026a), and synthetic environment generation is increasingly framed as a scalable alternative to manual benchmark curation (Wang et al., 2026; Dainese et al., 2024; Li et al., 2026a).

5.2.4 Co-evolution of world models and policies

The strongest integration arises when world models and policies co-evolve: the policy improves using model-generated experience, while the policy's new behaviors expose missing transitions that drive further model refinement. This is compelling for digital agents because interaction traces are often easy to log, replay, and verify. Recent work increasingly couples synthetic generation, model-based exploration, and policy updates iteratively (Fang et al., 2025; Feng et al., 2026; Wang et al., 2026; Xiao et al., 2026).

Takeaway The key design tension is not whether to use synthetic experience, but how tightly to couple model and policy. Open-loop augmentation is safe but passive: it cannot correct its own blind spots. Co-evolution is powerful but fragile: once the policy exploits systematic modeling errors, the feedback loop amplifies rather than corrects distributional drift. In practice, the strongest results appear in the middle of this spectrum, where the world model generates closed-loop experience but is periodically re-grounded against real interaction to prevent self-reinforcing hallucination.

5.3 Predictive Verification with World Models

A third usage pattern employs world models for **predictive verification**: assessing whether a candidate plan, assumption, or hypothesized trajectory is feasible, dynamically consistent, and safe under expected environment evolution (Figure 2, bottom right). Unlike planning, the goal is not to rank the next action. Unlike training-time interaction, it is not to construct additional experience. Instead, the world model projects likely consequences so that the agent can check feasibility, consistency, and risk before committing to execution (Gu et al., 2025; Qian et al., 2026; Zhao et al., 2026; Wang et al., 2025a; Chen et al., 2026).

This role is important because many digital-agent failures arise not from weak local action competence, but from incorrect assumptions about how the environment will evolve over multiple steps. Predictive verification

uses modeled futures to check whether the agent's beliefs, plans, and intended actions remain compatible with environment dynamics. This usage encompasses three closely related functions.

5.3.1 Hypothetical state evolution

The agent can propagate the consequences of its current assumptions: if a certain belief were correct, what states should follow. If a certain subgoal were pursued, what intermediate changes should appear. Such hypothetical evolution makes latent consequences explicit before execution, which is particularly useful in long-horizon digital tasks where errors often originate from incorrect intermediate assumptions rather than from a single poor local action (Gu et al., 2025; Wang et al., 2025a; Qian et al., 2026).

5.3.2 Consequence-based verification

Modeled consequences can support feasibility checking by testing whether a plan can execute under predicted dynamics, consistency checking by verifying that reasoning steps match anticipated state evolution, and counterfactual comparison by assessing whether alternative trajectories remain coherent under the same constraints. These forms of predictive verification are natural in digital environments with typed actions, hidden preconditions, and delayed constraints (Chae et al., 2025; Dagan et al., 2025; Mei et al., 2026; Guan et al., 2026; Zhao et al., 2026), helping distinguish fluent plans from dynamically valid ones.

5.3.3 Executable structure as a substrate for reliable checking

Digital environments often expose typed states, explicit action semantics, and structured failure signals, making modeled futures especially suitable for operational checks. World models can therefore help detect risky, irreversible, or policy-violating actions before execution (Chen et al., 2026; Guan et al., 2026; Feng et al., 2025a). The agent checks not merely whether a trajectory sounds plausible, but whether it satisfies the operational constraints of the environment it is about to act in.

Takeaway Planning tolerates occasional prediction errors because search can compare many rollouts. Verification is less forgiving, because a single missed precondition, invalid transition, or constraint violation may invalidate the conclusion. This asymmetry means that world models optimized for rollout generation, where aggregate statistical accuracy may suffice, can fail as verification modules, where individual predictions must remain semantically faithful to causal preconditions and operational constraints. Bridging this gap likely requires execution-grounded calibration or hybrid architectures that combine learned prediction with symbolic constraint checking.

5.4 Discussion

Recent systems increasingly combine these paradigms within a single architecture (Xiao et al., 2026; Fang et al., 2025; Wang et al., 2026; Guan et al., 2026; Deng et al., 2025), suggesting a broader shift toward treating world models as core cognitive infrastructure that links planning, learning, and verification (Ding et al., 2025; Bai et al., 2025; Feng et al., 2025a; Hao et al., 2023; Hu & Shu, 2023). The three paradigms also impose qualitatively different requirements on model fidelity: planning benefits from locally reliable and efficiently queryable predictions; training-time interaction emphasizes scalable rollout generation; and predictive verification demands predictions that are semantically meaningful enough to serve as evidence for or against candidate beliefs. A single predictive model may therefore not serve all three roles equally well.

6 Environments, Datasets, and Evaluation

This section treats environments, datasets, and evaluation as a coupled empirical pipeline. Environments determine which transition structure is actually present; datasets determine which portion of that structure becomes learnable; evaluation determines which claims about usefulness are empirically warranted. These

Table 3: Representative benchmarks for world models in digital-agent settings, organized by the primary predictive difficulty each environment introduces. Modality refers to the primary observation interface exposed to agents, where "State" denotes structured state or feature observations rather than raw pixels. Scale is reported in steps/frames (s), tasks (t), environments (e), or hours (h). "–" denotes open-ended settings.

Environment / Benchmark	Domain	Modality	Scale	Predictive Difficulty	Link
Atari 200M (Mnih et al., 2015)	Game, 2D	Visual	200M s	visual dynamics modeling under long-horizon control	GitHub
Atari 100k (Ye et al., 2021)	Game, 2D	Visual	100k s	sample-efficient rollout learning from limited data	GitHub
BabyAI (Chevalier-Boisvert et al., 2019)	Game, 2D	Text, Visual	19 e	grounded language-conditioned planning and hidden-state tracking	GitHub
SMAC (Samvelyan et al., 2019)	Game, 2D	State	14 e	multi-agent state evolution and coordination-sensitive rollout	GitHub
SMACv2 (Ellis et al., 2023)	Game, 2D	State	–	harder stochastic branching and robust multi-agent rollout	GitHub
Procgen (Cobbe et al., 2020)	Game, 2D	Visual	16 e	generalization under procedurally varying dynamics	GitHub
NetHack (Kuttler et al., 2020)	Game, 2D	Text, Visual	–	partial observability and long-horizon symbolic-visual state tracking	GitHub
MiniHack (Samvelyan et al., 2021)	Game, 2D	Text, Visual	–	modular long-horizon transition modeling across diverse tasks	GitHub
Crafter (Hafner, 2022)	Game, 2D	Visual	–	persistent survival-state dynamics and open-ended progression	GitHub
Minigrid (Chevalier-Boisvert et al., 2023)	Game, 2D	Visual	–	compact planning under partially observable gridworld dynamics	GitHub
PlanCraft (Dagan et al., 2025)	Game / Sandbox	Text, Visual	2,295 t	structured planning over branching crafting dependencies	GitHub
TextWorld (Côté et al., 2018)	Game, Text	Text	3,000 t	long-horizon symbolic transitions and command-conditioned dynamics	GitHub
ALFWorld (Shridhar et al., 2021)	Game, Text	Text	3,553 t	multi-step symbolic planning with delayed consequences	GitHub
ScienceWorld (Wang et al., 2022)	Game, Text	Text	30 t	causal reasoning over structured textual world state	GitHub
World of Bits (Shi et al., 2017)	GUI, Web	Text, Visual	–	open-domain web-state transitions under interface interaction	GitHub
MiniWoB++ (Liu et al., 2018)	GUI, Web	Text, Visual	114 e	short-horizon UI changes with typed web actions	GitHub
WebShop (Yao et al., 2022)	GUI, Web	Text	12,087 t	workflow branching and latent task progress in shopping interfaces	GitHub
WebArena (Zhou et al., 2024b)	GUI, Web	Text, Visual	812 t	semantic UI transitions under realistic web workflows	GitHub
VisualWebArena (Koh et al., 2024)	GUI, Web	Text, Visual	910 t	multimodal web-state prediction with visual-semantic grounding	GitHub
TheAgentCompany (Xu et al., 2025)	GUI, Web	Text, Visual	175 t	enterprise-style browser workflows and hidden application logic	GitHub
OSWorld (Xie et al., 2024)	GUI, Desktop	Text, Visual	369 t	open-ended computer-use transitions across apps and OS state	GitHub
WindowsAgentArena (Bonatti et al., 2025)	GUI, Desktop	Text, Visual	154 t	real multi-app state coordination under desktop interaction	GitHub
OmniACT (Kapoor et al., 2024)	GUI, Desktop	Text, Visual	9.8k t	executable computer-use actions with cross-platform UI effects	Website
AndroidWorld (Rawles et al., 2025)	GUI, Mobile	Text, Visual	116 t	dynamic mobile UI transitions under touch interaction	GitHub
MobileWorld (Kong et al., 2025b)	GUI, Mobile	Text, Visual	201 t	mobile workflows with user interaction and MCP-augmented tasks	GitHub
τ-bench (Yao et al., 2025)	Tool Use	Text	165 t	typed tool-state updates in tool-agent-user interaction	GitHub
τ^2-bench (Barres et al., 2025)	Tool Use	Text	279 t	multi-party control and persistent tool-state consistency	GitHub
τ^3-bench (Ray et al., 2026)	Tool Use	Text, Audio	278 t	voice-grounded tool use with multimodal state tracking	GitHub
BFCL v3 (Patil et al., 2025)	Tool Use	Text	1,000 t	multi-turn function-call validity and argument correctness	Website
MCP-Universe (Luo et al., 2025b)	Tool Use	Text, Visual	231 t	multi-server tool coordination and external-state mutation	GitHub
World of Workflows (Gupta et al., 2026)	Tool Use	Text	234 t	enterprise workflow transitions with hidden service-side dependencies	GitHub
AgentWorldModel-1K (Wang et al., 2026)	Tool Use	Text	1,000 e	synthetic external-state modeling in executable SQL-MCP ecosystems	GitHub
HumanEval (Chen et al., 2021)	Code	Text	164 t	executable outcome prediction for code generation	GitHub
SWE-bench (Jimenez et al., 2024)	Code	Text	2,294 t	repository-scale state evolution under code edits and tests	GitHub
SWE-bench Verified (Jimenez et al., 2024)	Code	Text	500 t	verified long-horizon software issue resolution with execution feedback	GitHub
InterCode (Yang et al., 2023)	Code	Text	5 e	interactive coding dynamics with stepwise execution feedback	GitHub
SWE-World (Sun et al., 2026b)	Code	Text	16.6k t	software-engineering transitions with surrogate environment feedback	GitHub
AgentGym (Xi et al., 2025)	Multi-domain	Text, Visual	14 e	transfer of predictive abstractions across heterogeneous environments	GitHub
AgentBench (Liu et al., 2024b)	Multi-domain	Text, Visual	1,284 t	cross-domain comparison of agent behavior and predictive utility	GitHub

layers do not align automatically. A benchmark may expose rich hidden-state dynamics while the dataset contains mostly nominal trajectories; a model may achieve strong local fidelity while failing under recursive rollout; an evaluation may reward perceptual plausibility while missing the sparse semantic distinction that actually determines success. The main empirical bottleneck is therefore rarely benchmark count alone, but whether environment richness, supervision richness, and evidence strength are aligned.

6.1 Interactive Environments and Benchmarks

Interactive environments are the primary empirical testbeds for world models because they expose closed-loop state transitions under sequential decision making. Table 3 summarizes representative benchmarks organized not as a flat domain inventory, but as a map of predictive difficulty regimes. Long horizons, hidden state, multi-agent branching, workflow compositionality, external-state persistence, and executable consequence are not interchangeable sources of difficulty. They favor different abstractions, different rollout strategies, and different notions of correctness. Benchmark choice should thus be treated as part of the modeling problem rather than as a purely empirical convenience.

Game and text-interactive environments Game-like digital environments remain valuable because they expose long horizons, delayed consequences, and relatively controlled transition dynamics. Text-based environments such as TextWorld (Côté et al., 2018), ALFWorld (Shridhar et al., 2021), and ScienceWorld (Wang et al., 2022) stress causal reasoning over symbolic or natural-language state. Although some are historically linked to embodied task families, their interaction interface is fully digital: the agent acts through discrete commands and receives structured textual state updates. Among more conventional game environments, Atari (Mnih et al., 2015; Ye et al., 2021), BabyAI (Chevalier-

Boisvert et al., 2019), SMAC/SMACv2 (Samvelyan et al., 2019; Ellis et al., 2023), Procgen (Cobbe et al., 2020), NetHack/MiniHack (Kuttler et al., 2020; Samvelyan et al., 2021), Crafter (Hafner, 2022), and Minigrid (Chevalier-Boisvert et al., 2023) provide complementary settings for studying hidden-state tracking, sample efficiency, procedural generalization, and rollout stability under limited interaction. PlanCraft (Dagan et al., 2025) is particularly relevant when the focus is explicit planning over structured digital tasks.

These environments test whether a model can maintain coherent dynamics over long horizons and remain stable under repeated imagined rollout. Their advantage is controllability. Their limitation is that they understate the interface heterogeneity and hidden service-side state that dominate web, GUI, and tool-use settings. Strong performance here is informative about long-horizon rollout but not sufficient evidence of robustness in broader digital-agent settings. We note that MineRL (Guss et al., 2019) and MineDojo (Fan et al., 2022) are excluded because their primary focus is physics-grounded embodied interaction.

GUI environments GUI environments expose a substantially different predictive regime. Agents interact through screenshots, DOM trees, accessibility structures, and typed interface actions, while much of the relevant state remains hidden behind application logic. Useful world models must therefore predict not only what the next interface looks like, but which semantically relevant interface changes follow from an action. Web benchmarks such as World of Bits (Shi et al., 2017), MiniWoB++ (Liu et al., 2018), WebShop (Yao et al., 2022), WebArena (Zhou et al., 2024b), VisualWebArena (Koh et al., 2024), and TheAgentCompany (Xu et al., 2025) require models to anticipate interface updates, workflow branching, and latent task progress. Desktop and computer-use benchmarks such as OSWorld (Xie et al., 2024), WindowsAgentArena (Bonatti et al., 2025), and OmniACT (Kapoor et al., 2024) further stress multi-application coordination and operating-system state, while AndroidWorld (Rawles et al., 2025) and MobileWorld (Kong et al., 2025b) highlight touch-based interaction and fast-changing UI dynamics on mobile devices.

Compared with game environments, GUI benchmarks place much greater pressure on partial observability and the mismatch between surface similarity and action-relevant correctness. A predicted screen may look plausible while being useless for control if it misses a modal dialog, a disabled button, or a hidden workflow transition. GUI environments are thus particularly informative for judging whether a world model captures the right abstraction of change: high observational richness does not imply high decision relevance.

Tool-use environments Tool-use environments shift the predictive difficulty. The main issue is not visible interface evolution, but action-conditioned mutation of external state through APIs, parameterized operations, database interactions, and multi-step workflows. The predictive target is often typed, schema-constrained, and only partially visible in the returned observation, making these environments especially suitable for symbolic or executable world modeling. τ-bench, τ^2-bench, and τ^3-bench (Yao et al., 2025; Barres et al., 2025; Ray et al., 2026) progressively extend this setting from tool-agent-user interaction to dual-control and voice-grounded tool use. BFCL v3 (Patil et al., 2025) focuses on function-calling evaluation, while MCP-Universe (Luo et al., 2025b), World of Workflows (Gupta et al., 2026), and AgentWorldModel-1K (Wang et al., 2026) move closer to realistic multi-server, enterprise-style, or synthetic service ecosystems.

These environments are valuable because they make transition semantics unusually explicit: the model must predict whether a call is valid, what state it mutates, and how one action constrains subsequent ones. At the same time, they also expose a recurring empirical weakness: tool benchmarks often provide strong end-task feedback but relatively thin supervision over latent external-state evolution, leaving evaluation overly focused on call correctness rather than persistent state tracking. As a result, tool-use benchmarks often measure whether the model understands interfaces to state change better than whether it can maintain state itself.

Code environments Code environments are especially informative for world-model research because they make functional correctness explicit. Actions correspond to program edits, command execution, or repository modification, and their consequences can often be checked directly through compilation, testing, or execution. HumanEval (Chen et al., 2021) evaluates generated programs by execution, while SWE-bench and SWE-bench Verified (Jimenez et al., 2024), InterCode (Yang et al., 2023), and SWE-World (Sun et al., 2026b) stress repository-scale reasoning, interactive coding, and long-horizon software-engineering workflows.

For world-model research, these benchmarks foreground executable semantics over surface similarity more clearly than any other domain. A useful model must predict not only what code might look like, but what effect it will have when run, providing perhaps the clearest testbed for verifiable consequence modeling. They also reveal an important asymmetry: code environments are often stronger on evaluation than on environment realism, because execution-based checking makes correctness measurable, but many tasks still compress the broader software process into relatively narrow issue-resolution setups. They are therefore excellent for studying process-level correctness and decision utility under executable feedback, but less complete as models of open-ended software ecosystems.

Multi-domain platforms Finally, multi-domain platforms test whether world-model-based agents can transfer across heterogeneous task structures. AgentGym (Xi et al., 2025) unifies diverse interactive environments spanning web, games, science tasks, tool use, and programming, while AgentBench (Liu et al., 2024b) evaluates agent behavior across multiple distinct environments. Their main value is diagnostic: they expose where predictive abstractions and usage patterns remain domain-specific and where they begin to generalize. However, their breadth can also obscure the specific predictive difficulty responsible for success or failure, which means they are most informative when paired with domain-specific analysis rather than treated as standalone proof of generality.

Takeaway The broader lesson from all environment families is that no single benchmark type is sufficient. Games are strong on controlled long-horizon rollout, GUI settings are strong on interface-mediated semantic change, tool-use environments are strong on typed external-state effects, and code environments are strong on executability and verifiability. A convincing empirical account of digital world models therefore requires not just cross-domain coverage, but explicit reasoning about which predictive difficulty each environment family contributes and how that difficulty should shape dataset design, model specification, and evaluation. Benchmark diversity matters not because more domains are always better, but because different domains reveal different failure modes of predictive structure.

6.2 Datasets for Learning Predictive Structure

If environments define what can in principle be interacted with, datasets define what can in practice be learned. The key issue is not simply scale, but which part of the environment's predictive structure becomes learnable. Datasets are never neutral samples: they select certain transitions, hide others, and often omit precisely the rare branches where predictive modeling is most useful. A useful organizing principle is the supervision object: local action effects, multi-step workflow regularities, execution-grounded correctness, typed tool semantics, broad symbolic priors, or synthetic branch expansion. These categories indicate what kind of predictive capability a dataset can support. Table 4 summarizes representative dataset types organized along this axis.

Action-conditioned transition datasets Many datasets provide explicit state-action-next-state supervision, making them the most direct substrate for learning environment dynamics. Examples include web interaction traces from Mind2Web (Deng et al., 2023), mobile and GUI trajectories from Android in the Wild and Android Control (Rawles et al., 2023; Li et al., 2024), and semantic world-modeling resources for mobile agents such as MobileWorldBench (Li et al., 2025a). These datasets are especially valuable because they expose local causal structure: what changes after a click, typed input, or interface action. Depending on the environment, the predictive target may be a DOM modification, a screenshot delta, a structured tool response, or a semantic state update, making them the natural data substrate for one-step transition modeling and semantic next-state prediction.

However, their limitations are equally important. Such datasets often cover only the realized action path and rarely record plausible alternatives from the same state. They also tend to overrepresent nominal transitions and underrepresent branching, failure, and recovery, making them strongest for learning local fidelity and weakest for supporting search, counterfactual imagination, or robust rollout under off-policy behavior. In

the language of Section 3, they are often well matched to narrow $\mathcal{X}$-level prediction targets, but insufficient by themselves for usage regimes $\mathcal{U}$ that require branching reasoning or long-horizon deliberation.

Trajectory demonstrations and behavioral priors Not all predictive structure is local. Many datasets instead provide longer trajectories that encode workflow regularities, high-level planning priors, and the temporal organization of interaction. Datasets such as GUI-360° (Mu et al., 2026), WebLINX (Lù et al., 2024), AMEX (Chai et al., 2025), and the broader Android GUI data resources discussed in (Li et al., 2024) contain structured demonstrations reflecting human workflows across digital environments. This category also includes digital game and text-interaction resources that expose sequential transition structure useful for world modeling. Atari-HEAD (Zhang et al., 2020) provides human demonstrations for visual game interaction. Text-interactive environments such as TextWorld (Côté et al., 2018), ALFWorld (Shridhar et al., 2021), and ScienceWorld (Wang et al., 2022) serve as trajectory sources for symbolic transition prediction and long-horizon reasoning. At larger scale, MineRL (Guss et al., 2019) offers approximately 60 million state-action pairs of human Minecraft gameplay, and VPT (Baker et al., 2022) provides over 70,000 hours of gameplay video for pretraining behavioral models.

These resources are important because they reveal not only what transition occurred, but how transitions compose over time, making them especially useful for learning workflow structure, hierarchical behavioral regularities, and long-horizon planning priors. Yet they also introduce a subtle risk: trajectory demonstrations may teach behavioral regularity without fully teaching environment dynamics. A model trained mostly on successful demonstrations may learn what good agents tend to do while remaining weak at predicting what happens under deviation, mistake, or rare branch exploration. Trajectory datasets are therefore indispensable for temporal abstraction but insufficient on their own for robust transition modeling. This distinction matters because agents do not fail only when they leave the demonstrated manifold; they often fail precisely because that manifold provides little evidence about what lies just outside it.

Execution-grounded supervision Code and program-like domains provide a stronger form of supervision: the consequence of an action can often be checked by execution itself. Datasets and benchmarks such as APPS (Hendrycks et al., 2021), MBPP (Austin et al., 2021), CodeXGLUE (Lu et al., 2021), and HumanEval (Chen et al., 2021) offer functional feedback through tests, compilation, or downstream task constraints. This form of supervision is particularly important for digital world modeling because it aligns prediction directly with functional correctness: a model trained under execution-grounded signals is encouraged to preserve executable semantics rather than surface similarity alone.

Their broader significance extends beyond code. Execution-grounded supervision demonstrates what a stronger empirical loop for world models could look like: instead of learning only from observed transitions, the model learns from whether its predicted consequences actually work. At the same time, execution-based data can be narrow in coverage, since pass/fail signals often say little about near-miss alternatives or latent causal structure unless paired with richer traces. Execution-grounded supervision is therefore unusually strong for correctness but not automatically strong for interpretability or broad transition coverage.

Tool interaction traces Tool-use datasets expose a complementary supervision object centered on typed actions and external-state effects. Representative examples include ToolBench (Qin et al., 2024), ToolAlpaca (Tang et al., 2023), and API-Bank (Li et al., 2023a). Compared with GUI datasets, these traces place less emphasis on perceptual state and more on schema-constrained arguments, tool semantics, and multi-step dependency structure, making them useful for learning symbolic transition models of external systems including validity constraints, API effects, and chained workflow logic.

Yet this category also illustrates a broader empirical gap: tool traces often teach how to issue calls, but only partially capture persistent external state across longer workflows. They are frequently adequate for evaluating local tool correctness, but less adequate for training or testing world models that must maintain hidden service-side state over many steps. In practice, they supervise action syntax and local effect validity more strongly than global state evolution.

Large-scale priors and foundation corpora Not all useful supervision is packaged as explicit transition tuples. Large-scale pretraining corpora can provide strong priors over code structure, interface conventions,

Table 4: Datasets for learning predictive structure in digital-agent environments. Dataset types are organized by the form of supervision they provide for modeling environment dynamics, workflow structure, and executable semantics. The final column highlights the main limitation of each supervision type, making explicit the gap between environment richness and supervision richness.

Supervision Type	Predictive Supervision and Limitation
Action-conditioned transitions	**Predictive structure:** state transitions and action effects. **Representative datasets:** Mind2Web (Deng et al., 2023), Android in the Wild (Rawles et al., 2023), Android Control / AITW (Li et al., 2024), MobileWorldBench (Li et al., 2025a). **Typical role in world models:** learning semantic UI dynamics, DOM / screenshot deltas, and action-conditioned next-state prediction. **Main limitation:** strong on local fidelity, but weak on branching alternatives, counterfactual transitions, and off-policy recovery.
Trajectory demonstrations and behavioral priors	**Predictive structure:** multi-step workflow structure, long-horizon interaction patterns, and sequential planning priors. **Representative datasets:** GUI-360° (Mu et al., 2026), WebLINX (Lù et al., 2024), AMEX (Chai et al., 2025), Android GUI data resources (Li et al., 2024), Atari-HEAD (Zhang et al., 2020), MineRL (Guss et al., 2019), VPT (Baker et al., 2022), TextWorld (Côté et al., 2018), ALFWorld (Shridhar et al., 2021), ScienceWorld (Wang et al., 2022). **Typical role in world models:** learning planning priors, capturing long-range workflow structure, and reducing long-horizon compounding error. **Main limitation:** captures successful behavioral regularities better than rare failures, deviations, or recovery trajectories.
Execution-grounded supervision	**Predictive structure:** functional correctness constraints and executable program semantics. **Representative datasets:** APPS (Hendrycks et al., 2021), MBPP (Austin et al., 2021), CodeXGLUE (Lu et al., 2021), HumanEval (Chen et al., 2021). **Typical role in world models:** learning constraint-aware prediction, execution-grounded transition modeling, and code-world semantics. **Main limitation:** strong correctness signal, but often sparse on intermediate causal traces, near-miss cases, and long-horizon branching structure.
Tool interaction traces	**Predictive structure:** tool semantics, API effects, and external-state mutation. **Representative datasets:** ToolBench (Qin et al., 2024), ToolAlpaca (Tang et al., 2023), API-Bank (Li et al., 2023a). **Typical role in world models:** modeling schema-constrained actions, external system dynamics, and multi-step tool workflows. **Main limitation:** often call-centric, with limited supervision over persistent hidden external state across longer workflows.
Large-scale priors and foundation corpora	**Predictive structure:** interface priors, code structure, and software semantics. **Representative datasets:** The Stack (Kocetkov et al., 2023), StarCoder Data (Li et al., 2023b). **Typical role in world models:** providing reusable semantic priors, improving representation quality, and supporting cross-domain generalization. **Main limitation:** provide broad semantic knowledge, but do not directly encode action-conditioned consequences.
Synthetic and simulated trajectories	**Predictive structure:** counterfactual transitions, branch expansion, and simulated interaction dynamics. **Representative datasets:** WebWorld (Xiao et al., 2026), Agent World Model (Wang et al., 2026). **Typical role in world models:** expanding branching coverage, surfacing rare or risky states, and improving planning robustness. **Main limitation:** can cover missing branches and failures, but risk simulator bias when synthetic futures drift from executable reality.

tool descriptions, and software semantics, which in turn improve the generalization of predictive models trained on smaller interactive datasets. Representative examples include The Stack (Kocetkov et al., 2023) and StarCoder Data (Li et al., 2023b). Although such corpora do not explicitly encode action-conditioned dynamics, they provide broad coverage of symbolic and procedural regularities that digital agents must reason over, often complementing transition-level data by supplying reusable representations of software artifacts, documentation, and executable structure.

From the standpoint of digital world modeling, their main role is to improve representation quality rather than directly provide transition supervision. They are therefore best understood as broad semantic priors that amplify, rather than replace, interaction-grounded data, which is why "more pretraining" and "better world modeling" should not be treated as synonymous.

Synthetic and simulated trajectories Because real interaction data is expensive, risky, and often sparse in failure or off-policy branches, recent work increasingly relies on synthetic trajectory generation and learned simulators. WebWorld (Xiao et al., 2026) shows that synthetic interaction traces can significantly improve downstream web-agent performance, while Agent World Model (Wang et al., 2026) pushes this idea toward large-scale synthetic environments for agentic reinforcement learning. Related directions in mobile and computer-use settings also suggest that synthetic data may become an increasingly important substrate for digital world modeling (Li et al., 2025a; Mu et al., 2026).

Synthetic data is particularly attractive because it directly targets one of the main weaknesses of logged traces: insufficient branching, failure, and off-policy coverage. In principle, it can approximate the counterfactual structure that real logs omit. But its usefulness depends entirely on whether the simulator preserves the decision-relevant properties of the underlying environment. If it fails to do so, synthetic data may scale supervision while simultaneously amplifying model bias. This creates a central tension: synthetic data is most valuable exactly where real data is weakest, but this is also where the risk of simulator-induced distortion is highest. The right question is therefore not whether synthetic data should be used, but what parts of the transition regime can be safely synthesized, which should remain grounded in real interaction, and how the two should be combined.

Takeaway Across all dataset types, the main empirical issue is not the absence of data, but a mismatch between environmental richness and supervision richness. Many environments define complex hidden-state, branching, and failure-sensitive dynamics, but the datasets sampled from them mainly expose successful local transitions. Current data resources are therefore often strongest exactly where world models are least needed, namely predicting nominal near-term change, and weakest where explicit predictive modeling is most valuable: long-horizon branching, rare failure, hidden constraint, and recovery dynamics. This asymmetry should guide both dataset construction and evaluation design. More sharply, the field's data bottleneck is not merely low coverage, but misallocated coverage.

6.3 Evaluation Protocols for World Models

Evaluating digital world models requires more than measuring one-step prediction quality. Because strong local fidelity does not necessarily translate into better planning, reasoning, or policy learning, evaluation should be understood as measuring decision-relevant predictive structure. At the same time, the empirical literature is unusually heterogeneous: reported gains often vary simultaneously with backbone strength, training data scale, interaction budget, rollout depth, planner design, and agent scaffold, making raw cross-paper comparison frequently much weaker evidence than it first appears. For digital agents, the key empirical question is often not "which paper reports the highest score?" but rather "under a fixed scaffold, does explicit predictive structure improve the same agent, for the same task, under the same budget?" We therefore organize evaluation along two complementary levels: core dimensions that define what kind of evidence a metric provides, and representative quantitative comparisons that distinguish settings where absolute comparison is meaningful from those where only setup-relative evidence is trustworthy.

6.3.1 Evaluation Dimensions

As summarized in Table 5, we organize evaluation along five complementary dimensions that form a layered evidence hierarchy: predictive fidelity (local transition accuracy), rollout consistency (multi-step coherence), process-level correctness (intermediate state validity), decision utility (downstream agent benefit), and efficiency and generalization (deployment reality). Each is most informative under different usage regimes, and stronger dimensions subsume weaker ones as evidence of practical value.

Predictive Fidelity Predictive fidelity measures how accurately a world model predicts environment transitions conditioned on actions. Because digital-agent world models may operate over pixels, DOM trees, accessibility structures, semantic state descriptions, tool outputs, latent states, or executable artifacts, the most informative intrinsic metric depends on the predictive target.

When the target is visual observation, common metrics include Mean Squared Error (MSE), Peak Signal-to-Noise Ratio (PSNR), Structural Similarity Index (SSIM), and Learned Perceptual Image Patch Similarity (LPIPS) (Wang et al., 2004; Zhang et al., 2018). For stochastic visual rollouts, Fréchet Inception Distance (FID) and Fréchet Video Distance (FVD) are often used to assess distributional realism and temporal coherence (Heusel et al., 2017; Unterthiner et al., 2018). These metrics remain useful for visual world models in games and GUI prediction, including recent generative GUI world models such as ViMo and MobileDreamer (Luo et al., 2025a; Cao et al., 2026).

In many digital-agent settings, however, semantic correctness matters more than perceptual similarity. When the target is a structured or semantic state, more informative metrics include Accuracy, Precision, Recall, and F1 for discrete state variables; IoU for predicted interface regions; API Call Accuracy for tool-use environments; and Pass Rate or execution success for code-related outputs. This shift is explicit in recent work. Web Agents with World Models argues for transition-focused abstraction, where the key predictive object is the semantically relevant state difference rather than the full HTML page (Chae et al., 2025). MobileWorldBench makes the same point by evaluating semantic next-state generation and future-state question answering for GUI agents (Li et al., 2025a).

The main lesson is that predictive fidelity should be matched to the transition semantics of the environment, not inherited mechanically from the representation modality. Visual similarity is meaningful when the task depends on full rendered state, but it is often weak evidence of usefulness in GUI, tool, or code settings where sparse semantic deltas or executable consequences dominate. A recurring weakness of current evaluation practice is therefore modality-driven metric choice: models are sometimes rewarded for predicting perceptually plausible but operationally irrelevant futures.

Rollout Consistency While predictive fidelity evaluates local transition accuracy, many agent tasks require coherent multi-step reasoning over imagined futures. Rollout consistency measures whether recursively composed predictions remain dynamically valid over multiple steps. In digital-agent environments, this corresponds to preserving executable validity and semantic coherence: GUI trajectories should maintain consistent interface structure across steps, tool calls should satisfy preconditions and argument constraints, code-edit trajectories should preserve executable semantics, and web workflows should follow valid task progression. Typical indicators include trajectory divergence rate, constraint violation frequency, invalid-action rate, logical contradiction frequency, and structural inconsistency across predicted states.

Rollout consistency is especially important when world models are used for planning, search, or training-time simulation. WebWorld evaluates long-horizon simulation quality for web-agent training and introduces dedicated multi-dimensional simulation metrics (Xiao et al., 2026). DynaWeb studies model-based reinforcement learning with web dynamics, where rollout quality matters directly for policy improvement (Ding et al., 2026). Computer-Using World Model and SafePred further emphasize stable multi-step environment reasoning under realistic computer-use constraints (Guan et al., 2026; Chen et al., 2026).

This dimension is also where the influence of data design becomes most visible. Datasets rich in one-step supervision but poor in branching, failure, or recovery coverage may support strong local accuracy while still producing brittle long-horizon rollouts. For that reason, rollout consistency often diagnoses not only model weakness, but also supervision weakness.

Decision Utility Ultimately, world models matter because they are used inside the agent loop. Decision utility measures whether predictive models improve downstream behavior, including planning quality, task success, reasoning reliability, or learning efficiency. In practice, it is measured through task-level performance such as Success Rate, Task Completion Rate, Episode Return, Solve Rate, Exact Match, or Pass@k in code generation. The central question is whether an agent with access to modeled futures performs better than a comparable agent without such access.

This distinction is particularly important in digital environments because useful prediction need not coincide with full reconstruction. A coarse semantic model may substantially improve behavior if it reliably captures action-relevant consequences, while a visually strong model may provide little practical benefit if it misses executable constraints, hidden workflow transitions, or safety-critical state changes. Representative examples include model-based web planning with LLMs (Gu et al., 2025), world-model-guided action selection in web navigation (Chae et al., 2025), synthetic-trajectory training in WebWorld (Xiao et al., 2026), model-based web-agent reinforcement learning in DynaWeb (Ding et al., 2026), code-based world models for search and decision support (Tang et al., 2024; Dainese et al., 2024), and world-model-enhanced reasoning for multi-turn VLM agents (Wang et al., 2025a).

Decision utility is the strongest evidence of practical benefit, but also the most confounded. Downstream performance depends not only on the world model, but also on the planner, policy, budget, and surrounding

Table 5: Evaluation dimensions and metrics for world models in digital-agent settings, organized as a layered evidence hierarchy from local transition accuracy to deployment-relevant agent benefit.

Evaluation Dimension	Metrics, Measured Property, and Best-Matched Usage
Predictive fidelity	**Representative metrics:** MSE, PSNR, SSIM, LPIPS, FID, FVD; Accuracy / Precision / Recall / F1; IoU; API Call Accuracy; execution success. **What it measures:** whether one-step predictions are correct under the target representation, including visual similarity, semantic state correctness, tool-call validity, or executable outcome prediction. **Best matched to:** one-step transition modeling, semantic delta prediction, local next-state forecasting, and target-specific intrinsic evaluation.
Rollout consistency	**Representative metrics:** trajectory divergence rate, constraint violation rate, invalid-action frequency, logical contradiction frequency, structural inconsistency. **What it measures:** whether recursively composed predictions remain dynamically valid, semantically coherent, and executable over multiple imagined steps. **Best matched to:** planning, search over imagined trajectories, training-time simulation, and long-horizon workflow tasks.
Decision utility	**Representative metrics:** Success Rate, Task Completion Rate, Episode Return, Solve Rate, Exact Match, Pass@k. **What it measures:** whether access to modeled futures improves planning, reasoning, policy learning, or end-task performance. **Best matched to:** agent-loop integration, decision-time planning benefit, synthetic-training benefit, and predictive-verification effectiveness.
Efficiency & generalization	**Representative metrics:** inference latency, memory footprint, parameter count, environment steps to threshold, zero-shot performance, cross-task transfer. **What it measures:** whether the world model is efficient enough for repeated online use and robust enough to generalize across new tasks, interfaces, layouts, or changing software environments. **Best matched to:** deployment-oriented evaluation, test-time scaling, repeated rollout use, interface drift, and cross-domain transfer.
Process-level correctness	**Representative metrics:** intermediate state validity, schema conformance, step-level semantic correctness, execution trace correctness, next-state QA / next-state generation accuracy. **What it measures:** whether intermediate predicted transitions are correct before final task completion, making it possible to diagnose where a model fails inside a trajectory. **Best matched to:** GUI, tool-use, and code environments with partially verifiable intermediate structure; diagnostic analysis of failure points.

system. A strong study should therefore connect end-task gains to lower evaluation layers: what predictive property improved, what rollout property was preserved, and why that translated into better behavior.

Efficiency and Generalization A useful world model must be efficient enough to query and robust enough to generalize. Efficiency metrics include inference latency, memory footprint, parameter count, and environment steps to achieve a target performance level, which matter especially when the model is invoked repeatedly during search, test-time scaling, or iterative reasoning. Generalization is equally important because digital environments are heterogeneous and evolving: useful evaluation should examine transfer across unseen tasks, new interfaces, updated layouts, modified rules, or different software ecosystems, particularly in web, GUI, and tool-use settings where interface drift can quickly invalidate learned predictors.

Recent work on large-scale simulators and synthetic environments makes this deployment perspective explicit. WebWorld studies cross-domain transfer beyond web simulation, while LLM-based simulators for evolving digital-agent training and OS-oriented neural simulators highlight scalability and transfer as first-class concerns (Xiao et al., 2026; Wang et al., 2025b; Rivard et al., 2026). Many current results are still reported in regimes where inference cost, rollout depth, or environment stationarity are artificially favorable. Yet explicit world models are worthwhile only if they remain queryable at the budgets where agents actually operate. Efficiency and generalization therefore serve as the main corrective against overvaluing models that are accurate but unusable, or useful only within a narrow frozen benchmark.

Process-Level Correctness A distinctive advantage of digital-agent settings is that intermediate states are often partially verifiable. GUI layouts can be checked against action effects, tool outputs validated against schemas, and code predictions compiled, executed, or tested. This suggests that digital world models should be evaluated not only by final outcomes, but also by the correctness of intermediate predicted transitions.

Process-level evaluation is especially important because it reveals where prediction fails: whether the model selects the wrong semantic delta, violates a tool constraint, or drifts into an inconsistent latent trajectory before final task failure becomes visible. MobileWorldBench evaluates semantic future-state prediction directly

rather than relying only on end-task reward (Li et al., 2025a). SafePred uses predictive modeling to detect risky future computer-use states before unsafe actions are executed (Chen et al., 2026). WorldCoder and related code-world-model work make intermediate executable reasoning itself observable through program execution and search feedback (Tang et al., 2024; Dainese et al., 2024).

For digital world modeling, this dimension is often the crucial bridge between intrinsic fidelity and downstream utility. A model may fail to improve final success either because it predicts the wrong local transition, because rollout drift accumulates, or because its predictions never expose the state distinctions that matter for action. Process-level correctness is uniquely valuable because it localizes which of these failures occurred. It is therefore not merely an auxiliary metric, but one of the clearest empirical advantages of studying explicit world models in software-mediated environments.

Takeaway These five dimensions form a layered evaluation framework rather than a flat metric checklist. Predictive fidelity is necessary but often weak evidence; rollout consistency strengthens the case for planning-oriented use; process-level correctness is especially diagnostic in digital settings; decision utility is the strongest but most confounded evidence of value; and efficiency plus generalization determine whether that value survives deployment reality. A mature empirical protocol should therefore ask four aligned questions: What property of the environment matters here? Does the available supervision actually teach it? Does the chosen metric reveal whether it was preserved? How strong is the resulting evidence claim? Under this view, evaluation is part of the same agent-centric alignment problem that structures the rest of the survey.

6.3.2 Representative Quantitative Comparisons Across Domains

Table 6 summarizes representative quantitative evidence across major digital-agent domains. Its purpose is not to produce a unified leaderboard, but to answer a more decision-relevant question: under what conditions does explicit predictive structure appear to help most clearly?

The answer depends on evidence strength. In a small number of relatively standardized settings, cross-paper absolute comparison is informative. In most web, GUI, tool-use, and code settings, however, reported results vary together with backbone strength, training data, interaction budget, planner design, and agent scaffold. For that reason, Table 6 distinguishes absolute results from setup-relative evidence. The former are useful for judging whether a world-model family is competitive under shared protocols. The latter are more useful for judging whether adding explicit predictive structure improves a reasonably fixed system.

Read in this way, the current evidence supports three practical conclusions.

Absolute evidence is strongest when the benchmark and budget are standardized. Atari 100k remains the clearest example: EfficientZero, DreamerV3, and IRIS (Ye et al., 2021; Hafner et al., 2025; Micheli et al., 2023) show that latent or tokenized predictive modeling can improve control under a shared evaluation regime. For practitioners, this is the closest the literature currently comes to saying that a world-model family is broadly effective in an apples-to-apples sense. The corresponding design lesson is that when the environment is stable, rollout-heavy, and repeatedly queried, efficient latent prediction remains the best-supported choice.

In language-mediated and GUI environments, the useful question is which predictive interface matches the consuming role. In text-interactive settings, results on ALFWorld from ReAct, RAFA, WALL-E 1.0, and WALL-E 2.0 (Yao et al., 2023b; Liu et al., 2024c; Zhou et al., 2024c; 2025), together with the consistency-ratio evidence of *From Word to World* on ALFWorld, ScienceWorld, and TextWorld (Li et al., 2026b), suggest that preserving coherent action-conditioned semantic consequence is more important than maximizing generic rollout depth. A similar pattern appears in GUI domains. WebDreamer, WAC, WebSynthesis, WebWorld, DynaWeb, ViMo, Code2World, MobileDreamer, SafePred, and CUWM (Gu et al., 2025; Shen et al., 2026; Gao et al., 2025; Xiao et al., 2026; Ding et al., 2026; Luo et al., 2025a; Zheng et al., 2026; Cao et al., 2026; Chen et al., 2026; Guan et al., 2026) do not support a clean cross-paper ranking of semantic, visual, or executable prediction. What they support instead is a role-conditional pattern: semantic

Table 6: Representative quantitative evidence for world models in digital-agent settings. This table is an evidence map, not a unified leaderboard. Single-value entries are absolute results. Entries written as "A vs. B" are setup-relative evidence (within-paper ablations or matched comparisons). For standardized settings such as Atari 100k, cross-paper comparison is meaningful; for web, GUI, tool-use, and code settings, results should be interpreted cautiously due to variation in backbone, scaffold, data, and protocol.

Domain	Method	Benchmark	Metric	Result	What the Evidence Most Strongly Supports
Evidence type: absolute single-result					
Game, 2D	EfficientZero (Ye et al., 2021)	Atari 100k	HNS (mean)	190	latent planning is highly effective under standardized repeated-control evaluation
Game, 2D	DreamerV3 (Hafner et al., 2025)	Atari 100k	HNS (mean)	125	latent imagination is competitive under shared rollout-heavy control
Game, 2D	IRIS (Micheli et al., 2023)	Atari 100k	HNS (mean)	105	tokenized predictive generation is viable under standardized visual control
Game, text	ReAct (Yao et al., 2023b)	ALFWorld	Success Rate	71	explicit deliberative simulation can improve long-horizon text acting
Game, text	RAFA (Liu et al., 2024c)	ALFWorld	Success Rate	95	long-horizon semantic planning is strongly supported in text-interactive worlds
Game, text	WALL-E 1.0 (Zhou et al., 2024c)	ALFWorld	Success Rate	95	rule-aligned semantic world modeling improves text-environment acting
Game, text	WALL-E 2.0 (Zhou et al., 2025)	ALFWorld	Success Rate	98	neuro-symbolic semantic alignment further strengthens text-world planning
Game, text	From Word to World (Li et al., 2026b)	ALFWorld	Consistency ratio	96	implicit text-world consistency can be measured directly and remains strong
Game, text	From Word to World (Li et al., 2026b)	ScienceWorld	Consistency ratio	91	implicit semantic consequence modeling generalizes to structured science tasks
Game, text	From Word to World (Li et al., 2026b)	TextWorld	Consistency ratio	92	implicit text-based world modeling remains coherent in symbolic game settings
Evidence type: setup-relative comparisons					
GUI, desktop	SafePred (Chen et al., 2026)	OS-Harm	Success Rate	35.0 vs. 22.0	outcome-oriented prediction improves safety-sensitive computer use
GUI, desktop	SafePred (Chen et al., 2026)	WASP	Success Rate	97.6 vs. 63.1	predictive guardrailing is highly effective when unsafe futures matter
GUI, desktop	CUWM (Guan et al., 2026)	GUI-360°	Accuracy	47.20 vs. 45.58	computer-use simulation yields measurable desktop prediction gains
GUI, web	WebDreamer (Gu et al., 2025)	VisualWebArena	Success Rate	23.6 vs. 17.6	semantic lookahead supports web planning under a fixed scaffold
GUI, web	WAC (Shen et al., 2026)	VisualWebArena	Success Rate	24.5 vs. 22.7	predictive correction helps local web action selection
GUI, web	WebEvolver (Fang et al., 2025)	WebVoyager	Success Rate	42.49 vs. 38.98	co-evolving policy and world model improves web interaction performance
GUI, web	DynaWeb (Ding et al., 2026)	WebVoyager	Success Rate	38.7 vs. 28.6	model-based RL can improve web-agent training when dynamics are learnable
GUI, web	WebSynthesis (Gao et al., 2025)	WebArena	Success Rate	14.93 vs. 9.70	search-guided synthesis helps planning-oriented web trajectory construction
GUI, web	WebWorld (Xiao et al., 2026)	WebArena	Success Rate	24.3 vs. 15.1	synthetic environment training can materially improve web-agent performance
GUI, mobile	ViMo (Luo et al., 2025a)	AndroidWorld	Success Rate	40.95 vs. 33.19	visual future generation supports mobile GUI acting
GUI, mobile	Code2World (Zheng et al., 2026)	AndroidWorld	Success Rate	50.86 vs. 41.38	renderable intermediate prediction supports stronger mobile control
GUI, mobile	MobileDreamer (Cao et al., 2026)	AndroidWorld	Success Rate	65.78 vs. 60.53	lightweight sketch-based prediction supports mobile rollout and control
Tool use	RWML (Yu et al., 2026)	τ^2-bench	Pass@1	43.7 vs. 38.0	predictive structure improves typed multi-turn tool interaction
Tool use	Agent World Model (Wang et al., 2026)	τ^2-bench	Pass@1	39.03 vs. 36.69	synthetic environments can reduce dependence on expensive real tool execution
Tool use	Agent World Model (Wang et al., 2026)	BFCLv3	Accuracy	70.18 vs. 61.25	explicit modeling helps function-calling correctness under controlled comparison
Tool use	Agent World Model (Wang et al., 2026)	MCP-Universe	Success Rate	12.29 vs. 8.38	predictive environments help with persistent multi-server tool coordination
Code	WorldCoder (Tang et al., 2024)	APPS	Pass@20	25.1 vs. 23.2	executable world modeling improves code search and solution generation
Code	GIF-MCTS (Dainese et al., 2024)	APPS	Pass@20	28.3 vs. 23.2	executable planning further improves code-generation quality
Code	GIF-MCTS (Dainese et al., 2024)	CWMB	Accuracy	84 vs. 79	search over executable futures supports stronger code-world reasoning
Code	SWE-World (Sun et al., 2026b)	SWE-bench Verified	Pass@1	55.0 vs. 54.6	surrogate environments provide modest software-engineering gains
Code	CWM (FAIR CodeGen team et al., 2025)	SWE-bench Verified	Pass@1	65.8 vs. 40.8	execution simulation yields strong gains under externally checkable software

predictors are most convincing for filtering and action correction, renderable or generative predictors for simulation and synthetic training, and outcome-oriented predictors for safety filtering and verification. For design purposes, the practical question in these domains is therefore: what future consequence does the downstream module actually need? The evidence favors matching the predictive interface to that need, rather than defaulting to the richest possible future representation.

Setup-relative evidence is strongest when predicted futures can be checked externally. In tool-use settings, RWML and Agent World Model report gains on τ^2-bench, BFCLv3, and MCP-Universe (Yu et al., 2026; Wang et al., 2026), suggesting that explicit predictive structure is particularly useful when tool effects are typed, persistent, and expensive to discover only through real execution. In code settings, WorldCoder and GIF-MCTS improve APPS, GIF-MCTS further improves CWMB, and execution-grounded systems such as SWE-World and CWM improve SWE-bench Verified (Tang et al., 2024; Dainese et al., 2024; Sun et al., 2026b; FAIR CodeGen team et al., 2025). The shared pattern is not merely that world models help in code, but that they help most clearly when imagined futures can be executed, tested, or otherwise validated before real commitment. For practitioners, this makes tool-use and code the most defensible regimes for investing in explicit predictive structure: external checking reduces the risk that the agent plans over attractive but invalid imagined futures.

Overall, Table 6 supports a qualified but practically useful selection rule. When protocols are standardized and repeated rollout matters, latent world models have the strongest absolute support. When the agent consumes predictions through filtering, correction, simulation, or guardrailing, the best-supported choice is usually the smallest predictive interface that matches that role. When futures can be executed, tested, or schema-checked, explicit world modeling has the clearest marginal value over purely reactive acting.

The broader implication is that quantitative evidence in this literature should be used less as a ranking device and more as a selection device. The most defensible conclusion at present is therefore conditional: explicit predictive structure can provide measurable benefit when its representational target, learning pathway, and usage role are appropriately matched to the surrounding agent scaffold, and when the evidence standard matches the strength of the claim being made.

6.4 Empirical Bottlenecks and Data Gaps

The main empirical bottleneck in digital world modeling is not the absence of benchmarks, datasets, or metrics in isolation. It is that these three layers are often misaligned. Benchmarks increasingly expose rich predictive difficulty, but datasets still emphasize nominal trajectories, and evaluation often rewards the easiest visible proxy rather than the consequence that matters most for downstream behavior.

This misalignment appears in three recurring forms.

Coverage remains concentrated on the easiest parts of the transition space. Current datasets disproportionately capture common or successful local changes while underrepresenting rare branches, hidden side effects, failures, and recovery dynamics (Guan et al., 2026; Gupta et al., 2026; Dhodapkar & Pishori, 2026; Hao et al., 2026). As a result, models are often best supervised exactly where explicit foresight is least needed, and weakest where it matters most.

Branch-sensitive prediction is still poorly supported. Logged traces usually record only the realized action path, not plausible alternatives from the same state. This weakens supervision for model-based search, counterfactual rollout, and recovery-aware reasoning (Cao et al., 2026; Dainese et al., 2024). Many current resources therefore supervise realized trajectories better than they supervise the reachable future space.

Evaluation is still too weakly coupled to the actual predictive difficulty of the environment. In GUI, tool-use, and other software-mediated settings, surface plausibility can diverge sharply from decision-relevant correctness. Yet process-level and branch-sensitive evaluation remain underused relative to their importance. This makes some empirical claims look stronger than the underlying evidence contract warrants.

Addressing these gaps will require richer online interaction, targeted synthetic data generation, and training pipelines that emphasize counterfactual, failure-focused, and recovery-oriented supervision (Betser et al., 2026; Cheng et al., 2025; Fang et al., 2025; Xiao et al., 2026; Wang et al., 2026). More fundamentally, the field needs end-to-end empirical design: the environment should expose the failure mode that matters, the dataset should cover it, and the evaluation protocol should justify the strength of the claim being made.

7 Open Problems and Future Directions

Despite rapid progress across all three dimensions of the design space introduced in Section 3, current systems remain far from providing reliable and general-purpose predictive infrastructure for digital agents. The central issue is no longer whether future consequences can be modeled at all, but whether the right predictive structure can be selected, learned, invoked, and validated under the constraints of real digital environments. The remaining open problems can therefore be organized around five questions: what should be modeled, how it should be trained, when it should be used, how it should be evaluated, and what kind of generality is actually realistic.

7.1 Representation Challenges: What Should Be Modeled?

The representation problem is fundamentally a problem of decision sufficiency. The best world model is rarely the one that predicts the most, but the one that preserves the smallest future description sufficient for downstream action. Current systems span all four prediction families from Section 4.1, yet principled criteria for choosing among them remain lacking.

The first unresolved issue is the trade-off among abstraction, recoverability, and verifiability. A compact representation improves efficiency, but may erase distinctions needed for debugging, recovery, or safety screening. A richer representation preserves more detail, but often increases rollout cost and brittleness. Executable interfaces partially reduce this tension, but are themselves hard to scale. A mature theory of digital world models will likely depend less on identifying one best modality than on characterizing which predictive interface remains simultaneously action-relevant, recoverable enough for downstream use, and externally checkable when verification matters.

The second unresolved issue is temporal abstraction. Many digital tasks are locally simple but globally brittle: the immediate next change may be easy to predict, while the longer causal chain that turns small early errors into failed workflows is much harder to represent. For many digital-agent tasks, the core representational challenge is therefore not next-state prediction alone, but preserving how local changes accumulate into persistent commitments, subgoals, and irreversible branches. Hierarchical predictive abstractions and long-range consequence modeling remain comparatively underdeveloped.

The third unresolved issue is unification. Different domains favor different predictive interfaces: structured deltas in GUI and web, typed state change in tool use, executability in code, and efficient latent rollout in games. The more plausible long-term target is therefore not a single universal predictive state, but a reusable grammar for selecting predictive interfaces under different environment and usage regimes. Representation-level generality may come less from one shared state space than from one shared set of design principles.

Two further gaps remain especially underexplored. Multi-agent digital settings weaken the assumption that the world is stationary, and uncertainty remains undertheorized even though many digital environments are partially observable, asynchronous, or externally dependent. The field still lacks a clear answer to a basic representational question: when is the right predictive object not one forecasted successor, but a structured set of plausible consequences?

7.2 Learning Challenges: How Can World Models Be Trained Efficiently and Reliably?

The central learning problem is not scale alone, but decision-relevant coverage under continual distribution shift. Offline data overrepresents nominal behavior, online interaction is expensive and risky, and synthetic data can amplify simulator bias. The result is a sharper bottleneck than simple data scarcity: digital world models are often least well trained on the futures where explicit foresight matters most.

The first challenge is consequence-weighted coverage. Rare but important transitions, including UI failures, hidden backend mutations, schema mismatches, unsafe actions, and recovery branches, are systematically underrepresented in existing data. Future pipelines therefore need to optimize not merely for average transition coverage, but for coverage of the consequences agents cannot afford to misunderstand.

The second challenge is continual drift. Layouts change, APIs evolve, tools expand, dependencies shift, and the policy itself induces new state distributions. In deployed digital settings, continual adaptation is not merely an optional refinement of world modeling. It increasingly becomes part of the problem definition. At the same time, adaptation cannot come at unlimited cost, which means selective prediction, adaptive rollout depth, and modular predictive components are likely to become part of the learning problem rather than only inference-time engineering.

The third challenge is objective mismatch. Predictive losses often reward local plausibility or average one-step accuracy, while agent failure is driven by sparse semantic errors, missed preconditions, and broken branching structure. The key objective-design question is therefore not just how to reduce average prediction error, but how to concentrate modeling accuracy on the errors agents most need to avoid. This points toward verifier-grounded objectives, counterfactual supervision, decision-conditioned reweighting, and explicit penalties on unrecoverable or constraint-violating mistakes.

Finally, learning cannot be separated from the policy that consumes the model. Even a strong model may rapidly become unreliable once the policy visits new parts of the state space. The broader problem is thus to maintain decision-relevant predictive support under simultaneous environment drift, policy drift, and coverage asymmetry.

7.3 Usage Challenges: How Should World Models Support Agent Behavior?

The central usage problem is that world-model quality is role-conditional. A predictor that is adequate for one-step filtering may fail for long-horizon search, and a model that supports rollout generation may still be unusable as a verification module. The field therefore still lacks principled criteria for matching predictive fidelity to agent-facing role.

The first implication is that there is unlikely to be a single notion of predictive quality that is simultaneously optimal for filtering, planning, verification, and recovery. Short-horizon decision support mainly requires cheap local discrimination. Verification requires semantic faithfulness and constraint preservation. Safety screening requires calibrated and often conservative forecasts. Recovery planning requires counterfactual support over alternatives. The operative question is therefore not whether a world model is good in the abstract, but good for which role inside the agent loop.

The second challenge is invocation policy. In practice, world models interact with foundation models, retrieval, memory, tool execution, external verifiers, and safety filters. Their practical value may depend as much on when they are called as on how accurate they are. A highly accurate model can still have low value if invoked at the wrong time, while a weaker one may be useful if called only when task horizon, uncertainty, branching factor, or failure cost justify explicit foresight.

This sharpens the cost question. Explicit world models are unlikely to be uniformly worthwhile. They are most justified when actions are hard to reverse, mistakes are expensive, constraints are hidden, or recovery is difficult. For easier settings, reactive policies or strong foundation models with implicit transition knowledge may already suffice. A mature usage theory must therefore explain not only how to use world models, but where their additional cost is actually warranted.

Two especially promising directions are safety and self-regulation. Safety-aware world modeling concerns when predictive support should block, delay, or qualify an action under uncertainty. Self-regulatory world modeling concerns how predictive structure can support diagnosis, rollback, re-grounding, and recovery after deviation. In that sense, the highest-value use of world models may be not only to improve already-good trajectories, but to make bad trajectories diagnosable, interruptible, and recoverable.

7.4 Evaluation Challenges

Evaluation remains unsettled not because the field lacks metrics, but because it still lacks stable evaluation contracts. Current work reports predictive fidelity, rollout consistency, process-level correctness, decision utility, and cost, but much less often specifies what kind of evidence each metric is intended to provide for the model's role. The unresolved issue is therefore evidence alignment.

The first challenge is the gap between intrinsic and extrinsic evidence. High local accuracy does not imply downstream usefulness, and end-task gains do not cleanly identify which predictive property improved. Future evaluation must therefore explain not only whether a world model helps, but which preserved predictive property caused the behavioral gain. This makes process-level diagnosis and rollout-level validity especially important in digital settings, where intermediate states are often partially verifiable.

The second challenge is comparability. Results currently differ simultaneously in backbone, training data, interaction budget, planner depth, agent scaffold, and benchmark protocol, making many cross-paper comparisons only weakly informative. The field needs more standardized and usage-aware protocols that isolate the marginal value of explicit predictive structure rather than rhetorically merging fundamentally different setups. At the same time, benchmark realism must improve: future protocols should better expose interface drift, persistent external state, long-horizon branching, and safety-sensitive failures.

The third challenge is cost-aware reporting. Many world models improve results by increasing inference-time deliberation, but final task score alone does not reveal whether the gain reflects better predictive structure or simply more compute. For digital-agent world models, performance without rollout budget, query count, or wall-clock cost is incomplete evidence.

Finally, evaluation should be explicitly usage-conditional. A verification-oriented model should not be judged as if it were a generic rollout generator, and a one-step filtering model should not be penalized for lacking realism it was never intended to provide. A mature protocol should therefore answer three aligned questions: what role is the model intended to play, what evidence layer is strong enough to justify that role, and at what cost does the claimed benefit appear?

7.5 Toward General-Purpose World Models for Digital Agents

A natural long-term ambition is to develop world models that support agents across games, web and GUI environments, tool-use ecosystems, and code tasks. Yet the survey also suggests that this goal is easy to misstate. The central question is not whether one monolithic predictor can cover all digital environments, but what kind of generality is actually realistic.

The first proposition is that general-purpose world modeling should not be equated with one universal state representation. Different domains expose different notions of state, transition, and correctness, and forcing them too early into a single predictive interface may reduce rather than increase usefulness. A more realistic target is a reusable design grammar: a principled way to choose predictive interface, learning pathway, and usage role under different digital environments.

The second proposition is that explicit world models and foundation models are more likely to be complementary than mutually exclusive. Foundation models provide broad priors, flexible reasoning, and zero-shot pattern completion. Explicit world models provide grounded, action-conditioned, and externally testable consequence interfaces. The main bottleneck is therefore increasingly one of system-level coupling: when to invoke explicit prediction, how to combine it with retrieval or verification, and how to act under uncertain or conflicting forecasts.

The third proposition is that near-term progress may come faster from systems-level and evaluative unification than from full representational unification. Standardizing how world models are integrated, queried, and compared may be more tractable than forcing agreement on one universal predictive state. In that sense, the path to broader generality may proceed first through better interfaces, better coupling, and better evidence, and only later through deeper representational unification.

Ultimately, the goal is not prediction for its own sake, but predictive structure that makes digital agents more deliberate, data-efficient, verifiable, and robust. Whether world models become a central organizing principle will depend less on finding one dominant model family than on aligning prediction, grounding, and usage across heterogeneous digital environments.

7.6 Design Guidelines from the Agent-Centric Design Space

The design space $\mathcal{W} = (\mathcal{X}, \mathcal{L}, \mathcal{U})$ is useful not only as a taxonomy, but as a practical design language for deciding when explicit world modeling is warranted, what form it should take, and how it should be integrated into the agent loop. The broader lesson of this survey is that effective digital-agent world models are rarely obtained by maximizing predictive richness in isolation. They emerge when model specification $\mathcal{X}$, learning and system realization $\mathcal{L}$, and agent-facing usage $\mathcal{U}$ are jointly matched to environment structure, failure cost, observability, branching factor, and the role predictive support is meant to play.

This yields one general principle: world-model design should begin from failure mode and usage regime, not from architecture alone. The practical question is not which world-model family is best in the abstract, but which predictive interface is sufficient for the specific distinction the agent cannot afford to miss, which learning signal can support that interface, and under what conditions the resulting system is worth its modeling and inference cost.

We summarize this perspective as six design guidelines.

Guideline 1: Predict the smallest future that preserves the distinction the agent cannot afford to miss. In digital environments, more prediction is not automatically better prediction. When the downstream decision depends mainly on sparse semantic changes, such as whether a button becomes enabled, a dialog appears, a tool call mutates external state, or an edit violates a constraint, state-delta or auxiliary-outcome prediction is often preferable to full future reconstruction. Full-observation prediction is best reserved for settings where the relevant consequence is diffuse, hard to localize, or tightly coupled to layout-level evolution. The key design question is therefore not how much future can be predicted, but what is the minimal sufficient future for the intended usage regime.

Guideline 2: Match the architecture to the dominant failure mode, not merely to the input modality. Architectural choice should be driven by what kind of mistake is most costly. LLM-based predictors are often natural when the core difficulty is symbolic, workflow-level, or naturally expressible as semantic state change, as in semantic web-state forecasting and action-conditioned consequence prediction for browser agents (Gu et al., 2025; Chae et al., 2025). Visual generative models are strongest when action depends directly on rendered state, layout evolution, or appearance-grounded affordances (Luo et al., 2025a; Cao et al., 2026). Latent dynamics are attractive when rollout efficiency is the bottleneck and deep imagination is needed (Hafner et al., 2020; 2025; Micheli et al., 2023). Code-based or executable models are especially valuable when predicted futures can be rendered, executed, or externally checked, as illustrated by renderable GUI prediction and executable environment construction (Tang et al., 2024; Zheng et al., 2026). In heterogeneous settings, hybrid systems are often the most defensible choice, because the dominant failure is frequently not missing one modality, but failing to align semantic abstraction, efficient rollout, and external validation inside a single agent loop (Mei et al., 2026; Shen et al., 2026).

Guideline 3: Let the usage regime determine how much fidelity is necessary. Predictive fidelity should be defined relative to role, not in the abstract. For short-horizon filtering, cheap one-step semantic prediction may suffice (Gu et al., 2025; Chae et al., 2025). For long-horizon search, rollout stability, branch discrimination, and error containment matter more than high-detail reconstruction (Tang et al., 2024; Mei et al., 2026). For predictive verification and safety screening, semantic faithfulness and constraint preservation matter more than average-case realism (Wang et al., 2025a; Chen et al., 2026). A model that is entirely adequate for action pruning may still be too weak for verification, while a model designed for verification may be unnecessarily expensive for local filtering. The right question is therefore not whether a model is accurate in general, but whether it is accurate in the way required by its role in $\mathcal{U}$.

Guideline 4: Expand supervision where logged traces are weakest, not where they are already abundant. Offline interaction logs are typically strongest on nominal transitions and weakest on branching, failure, deviation, and recovery. Yet these underrepresented regions are often exactly where anticipatory modeling matters most. Online interaction, counterfactual expansion, and synthetic generation are therefore most valuable when they fill decision-critical holes in coverage rather than simply reproduce the observed distribution at larger scale. This pattern is increasingly visible in large-scale synthetic web simulation, model-based agent training, and policy–world-model co-improvement pipelines (Xiao et al., 2026; Ding et al., 2026; Fang et al., 2025; Wang et al., 2026). The operative objective is not more transition data in general, but better support over the futures the agent is least able to recover from misunderstanding. From this perspective, learning digital world models is fundamentally a problem of decision-aligned coverage construction.

Guideline 5: Treat uncertainty as part of the predictive interface when hidden state or branching ambiguity is substantial. Many digital environments contain latent variables that are only partially revealed through interface observations: server-side state, authentication context, hidden tool state, repository dependencies, or delayed workflow constraints. In such settings, a point prediction can be actively misleading. When branching uncertainty is large, the model should expose ambiguity rather than collapse it, for example through multiple candidate futures, confidence-aware pruning, retrieval support, or explicit downstream verification (Mei et al., 2026; Shen et al., 2026). This is especially important for long-horizon planning and safety-sensitive settings, where overconfident but weakly grounded predictions can be more damaging than limited foresight.

Guideline 6: Explicit world models are most justified when failure is costly and reactive correction is hard. The marginal value of explicit world modeling is highest when tasks are long-horizon, branch-sensitive, partially observable, hard to recover from, or externally verifiable. Their benefit is lower when tasks are short-horizon, easily reversible, weakly branching, or already well served by strong foundation-model priors plus lightweight deliberation. Recent work on predictive safety guardrails illustrates the value of explicit foresight when delayed and irreversible risk matters, while negative evidence also shows that explicit models do not help automatically unless agents know how to use them effectively (Chen et al., 2026; Qian et al., 2026). The key systems decision is therefore not whether digital agents should use explicit world

models in general, but whether the expected reduction in consequential errors exceeds the added cost of model construction, training, coupling, and inference.

These guidelines suggest a compact decision procedure. A practitioner should first identify the dominant failure source: missing local semantic change, misranking long-horizon branches, violating hidden constraints, or failing to verify execution-level consequences. They should then choose the smallest predictive interface in $\mathcal{X}$ that makes this failure visible, the strongest grounding signal in $\mathcal{L}$ that can supervise it, and the narrowest agent-facing role in $\mathcal{U}$ that can actually consume it. Only then should architecture be chosen. This procedure is intentionally simple, but it captures the central practical implication of the agent-centric view: world-model design should be driven by failure structure, evidence availability, and usage role before it is driven by architectural fashion.

Conditional recommendations by task regime. The design space also supports coarse but useful conditional recommendations. When tasks are short-horizon, locally observable, and largely reversible, a lightweight semantic or state-delta predictor paired with short-horizon lookahead is often the most cost-effective choice (Gu et al., 2025; Chae et al., 2025). When tasks are long-horizon, branch-sensitive, and difficult to recover from, latent or hybrid models coupled with search-based planning or model-assisted training become more justified (Tang et al., 2024; Mei et al., 2026; Ding et al., 2026). When the primary risk is not poor search but invalid assumptions, hidden constraints, or costly irreversible actions, verification-oriented designs are preferable: the world model should expose semantically faithful, constraint-aware, and ideally externally checkable consequences (Chen et al., 2026; Zheng et al., 2026). When environments are highly heterogeneous, for example combining GUI state, tool outputs, and executable effects, hybrid realizations are often more robust than forcing a single predictive interface to absorb all burdens simultaneously (Mei et al., 2026; Shen et al., 2026). These recommendations are deliberately conditional rather than universal: the point of $\mathcal{W} = (\mathcal{X}, \mathcal{L}, \mathcal{U})$ is not to prescribe one best design, but to make the trade-offs explicit.

Illustrative $\mathcal{X}$–$\mathcal{L}$–$\mathcal{U}$ mismatch diagnosis. A recurring failure pattern in current systems is not weak prediction per se, but misalignment among what is predicted, how it is learned, and how it is used.

A first mismatch is over-rich prediction for a weak usage role. For example, full-observation or render-heavy future prediction may be paired with short-horizon action filtering, where the agent mainly needs to know whether a control becomes available, whether a dialog appears, or whether a tool call triggers a constraint violation. In such cases, the system pays the cost of high-coverage prediction without fully exploiting the extra information. This helps explain why semantically targeted predictors can outperform visually richer alternatives when the downstream role is narrow and local (Gu et al., 2025; Chae et al., 2025).

A second mismatch is under-grounded prediction for a strong usage role. Lightweight semantic predictors may be used as if they were verification modules, even though verification requires stronger preservation of preconditions, constraints, and executable consequences than filtering does. A predictor that is adequate for ranking local actions may still be too weak to validate a multi-step plan or certify safety. Recent predictive safety and executable world-modeling work illustrates this gap (Chen et al., 2026; Zheng et al., 2026).

A third mismatch is strong prediction with weak coupling. A world model may be locally competent, yet produce little downstream gain because the agent queries it too rarely, searches over it too shallowly, ignores its uncertainty, or lacks a mechanism for re-grounding imagined futures against real execution. This pattern is consistent with recent findings that current agents often fail to strategically leverage world models as foresight tools, and with evidence that retrieval grounding can partially repair long-horizon degradation (Qian et al., 2026; Mei et al., 2026). Under this view, many negative empirical results are better interpreted not as evidence that explicit world modeling is intrinsically ineffective, but that the selected $(\mathcal{X}, \mathcal{L}, \mathcal{U})$ combination is poorly aligned with the actual failure mode and evidence budget of the task.

When are explicit world models worth their cost? The emerging boundary is best understood as a conditional claim rather than a universal one. Explicit world models are most likely to provide positive marginal value when tasks are long-horizon, irreversible, branch-sensitive, partially observable, or amenable to external checking (Chen et al., 2026; Zheng et al., 2026). Their advantage is less stable when tasks are local, reactive, easily reversible, or already covered by strong foundation-model priors and modest inference-

time deliberation (Qian et al., 2026). This suggests that the central research question is not whether explicit world models can help digital agents, but when explicit predictive structure improves consequential decisions enough to justify its additional construction and inference burden. In other words, the right comparison is rarely world model versus no world model in the abstract. It is explicit predictive structure versus simpler reactive or foundation-model-based alternatives under a fixed failure tolerance and compute budget.

8 Conclusion

This survey introduced the agent-centric design space $\mathcal{W} = (\mathcal{X}, \mathcal{L}, \mathcal{U})$ for understanding world models in digital agents. Our central claim is that world models in these settings are best understood as agent-facing predictive infrastructures whose value depends on the alignment among what is modeled, how it is built, and what role it serves inside the agent loop. No single world-model family is uniformly dominant. Explicit predictive structure is most useful when its predictive interface, grounding signal, and usage regime are well matched to the surrounding task and system. The main obstacles are now clear: datasets remain weak on consequential branches; evaluation still often measures convenient proxies more faithfully than downstream utility; and many failures arise from poor coupling between predictive modules and the agent that consumes them. The path forward is less about searching for one universal predictor than about developing reliable, cost-aware, and usage-conditional principles for deciding what should be modeled, how it should be learned, when it should be invoked, and how its value should be verified.

References

Mayank Agarwal, Ibrahim Abdelaziz, Kinjal Basu, Merve Unuvar, Luis A. Lastras, Yara Rizk, and Pavan Kapanipathi. Toolrm: Outcome reward models for tool-calling large language models. *ArXiv*, abs/2509.11963, 2025. URL https://api.semanticscholar.org/CorpusID:281314876.

Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie J. Cai, Michael Terry, Quoc V. Le, and Charles Sutton. Program synthesis with large language models. *ArXiv*, abs/2108.07732, 2021. URL https://api.semanticscholar.org/CorpusID:237142385.

Jinbin Bai, Yu Lei, Hecong Wu, Yuchen Zhu, Shufan Li, Yi Xin, Xiangtai Li, Molei Tao, Aditya Grover, and Ming-Hsuan Yang. From masks to worlds: A hitchhiker's guide to world models. *ArXiv*, abs/2510.20668, 2025. URL https://api.semanticscholar.org/CorpusID:282304134.

Bowen Baker, Ilge Akkaya, Peter Zhokhov, Joost Huizinga, Jie Tang, Adrien Ecoffet, Brandon Houghton, Raul Sampedro, and Jeff Clune. Video pretraining (vpt): Learning to act by watching unlabeled online videos. In *Advances in Neural Information Processing Systems*, 2022. URL https://api.semanticscholar.org/CorpusID:249953673.

Victor Barres, Honghua Dong, Soham Ray, Xujie Si, and Karthik Narasimhan. τ2-bench: Evaluating conversational agents in a dual-control environment. *ArXiv*, abs/2506.07982, 2025. URL https://api.semanticscholar.org/CorpusID:279251284.

Marc G. Bellemare, Yavar Naddaf, Joel Veness, and Michael Bowling. The arcade learning environment: An evaluation platform for general agents. *Journal of Artificial Intelligence Research*, 47:253–279, 2013. URL https://api.semanticscholar.org/CorpusID:1552061.

Roy Betser, Shamik Bose, Amit Giloni, Chiara Picardi, Sindhu Padakandla, and Roman Vainshtein. Agentrim: Tool risk mitigation for agentic ai. *ArXiv*, abs/2601.12449, 2026. URL https://api.semanticscholar.org/CorpusID:284909839.

Rogerio Bonatti, Dan Zhao, Francesco Bonacci, Dillon Dupont, Sara Abdali, Yinheng Li, Yadong Lu, Justin Wagle, Kazuhito Koishida, Arthur Fender C. Bucker, Lawrence Jang, and Zack Hui. Windows agent arena: Evaluating multi-modal os agents at scale. In *International Conference on Machine Learning*, 2025. URL https://api.semanticscholar.org/CorpusID:272600411.

Jake Bruce, Michael Dennis, Ashley Edwards, Jack Parker-Holder, Yuge Shi, Edward Hughes, Matthew Lai, Aditi Mavalankar, Richie Steigerwald, Chris Apps, Yusuf Aytar, Sarah Bechtle, Feryal Behbahani, Stephanie Chan, Nicolas Heess, Lucy Gonzalez, Simon Osindero, Sherjil Ozair, Scott Reed, Jingwei Zhang, Konrad Zolna, Jeff Clune, Nando de Freitas, Satinder Singh, and Tim Rocktäschel. Genie: Generative interactive environments. In *International Conference on Machine Learning*, 2024. URL https://api.semanticscholar.org/CorpusID:267897982.

Yilin Cao, Yufeng Zhong, Zhixiong Zeng, Liming Zheng, Jing Huang, Haibo Qiu, Peng Shi, Wenji Mao, and Guanglu Wan. Mobiledreamer: Generative sketch world model for gui agent. In *International Conference on Learning Representations*, 2026. URL https://api.semanticscholar.org/CorpusID:284532168.

Hyungjoo Chae, Namyoung Kim, Kai Tzu iunn Ong, Minju Gwak, Gwanwoo Song, Jihoon Kim, Sunghwan Kim, Dongha Lee, and Jinyoung Yeo. Web agents with world models: Learning and leveraging environment dynamics in web navigation. In *International Conference on Learning Representations*, 2025. URL https://api.semanticscholar.org/CorpusID:273404026.

Yuxiang Chai, Siyuan Huang, Yazhe Niu, Han Xiao, Liang Liu, Dingyu Zhang, Shuai Ren, and Hongsheng Li. Amex: Android multi-annotation expo dataset for mobile gui agents. In *Findings of the Association for Computational Linguistics*, 2025. URL https://api.semanticscholar.org/CorpusID:271432257.

Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. Evaluating large language models trained on code. *ArXiv*, abs/2107.03374, 2021. URL https://api.semanticscholar.org/CorpusID:235755472.

Yurun Chen, Zeyi Liao, Ping Yin, Taotao Xie, Keting Yin, and Shengyu Zhang. Safepred: A predictive guardrail for computer-using agents via world models. *ArXiv*, abs/2602.01725, 2026. URL https://api.semanticscholar.org/CorpusID:285269046.

Yuhao Cheng, Liang Tang, Shuxian Li, Yukang Huo, Tiaonan Duan, Kaer Huang, Yanzhe Jing, and Yiqiang Yan. Evolving in tasks: Empowering the multi-modality large language model as the computer use agent. *ArXiv*, abs/2508.04037, 2025. URL https://api.semanticscholar.org/CorpusID:280536247.

Maxime Chevalier-Boisvert, Dzmitry Bahdanau, Salem Lahlou, Lucas Willems, Chitwan Saharia, Thien Huu Nguyen, and Yoshua Bengio. Babyai: A platform to study the sample efficiency of grounded language learning. In *International Conference on Learning Representations*, 2019. URL https://api.semanticscholar.org/CorpusID:59536625.

Maxime Chevalier-Boisvert, Bolun Dai, Mark Towers, Rodrigo de Lazcano, Lucas Willems, Salem Lahlou, Suman Pal, Pablo Samuel Castro, and Jordan K. Terry. Minigrid & miniworld: Modular & customizable reinforcement learning environments for goal-oriented tasks. In *Advances in Neural Information Processing Systems*, 2023. URL https://api.semanticscholar.org/CorpusID:259252185.

Meng Chu, Xuan Billy Zhang, Kevin Qinghong Lin, Lingdong Kong, Jize Zhang, Teng Tu, Weijian Ma, Ziqi Huang, Senqiao Yang, Wei Huang, Yeying Jin, Zhefan Rao, Jinhui Ye, Xinyu Lin, Xichen Zhang, Qisheng Hu, Shuai Yang, Leyang Shen, Wei Chow, Yifei Dong, Fengyi Wu, Quanyu Long, Bin Xia, Shaozuo Yu, Mingkang Zhu, Wenhu Zhang, Jiehui Huang, Haokun Gui, Haoxuan Che, Long Chen, Qifeng Chen, Wenxuan Zhang, Wenya Wang, Xiaojuan Qi, Yang Deng, Yanwei Li, Mike Zheng Shou, Zhi-Qi Cheng, See-Kiong Ng, Ziwei Liu, Philip Torr, and Jiaya Jia. Agentic world modeling: Foundations, capabilities, laws, and beyond. *ArXiv*, abs/2604.22748, 2026. URL https://arxiv.org/abs/2604.22748.

Karl Cobbe, Christopher Hesse, Jacob Hilton, and John Schulman. Leveraging procedural generation to benchmark reinforcement learning. In *International Conference on Machine Learning*, 2020. URL https://api.semanticscholar.org/CorpusID:208547624.

Marc-Alexandre Côté, Ákos Kádár, Xingdi Yuan, Ben Kybartas, Tavian Barnes, Emery Fine, James Moore, Ruo Yu Tao, Matthew Hausknecht, Layla El Asri, Mahmoud Adada, Wendy Tay, and Adam Trischler. Textworld: A learning environment for text-based games. In *CGW@IJCAI*, 2018. URL https://api.semanticscholar.org/CorpusID:49552345.

Gautier Dagan, Frank Keller, and Alex Lascarides. Plancraft: an evaluation dataset for planning with llm agents. In *Conference on Language Modeling*, 2025. URL https://api.semanticscholar.org/CorpusID:275133533.

Nicola Dainese, Matteo Merler, Minttu Alakuijala, and Pekka Marttinen. Generating code world models with large language models guided by monte carlo tree search. In *Advances in Neural Information Processing Systems*, 2024. URL https://api.semanticscholar.org/CorpusID:270045176.

Mingkai Deng, Jinyu Hou, Zhiting Hu, and Eric P. Xing. Simura: A world-model-driven simulative reasoning architecture for general goal-oriented agents. *ArXiv*, abs/2507.23773, 2025. URL https://api.semanticscholar.org/CorpusID:280400991.

Xiang Deng, Yu Gu, Boyuan Zheng, Shijie Chen, Samuel Stevens, Boshi Wang, Huan Sun, and Yu Su. Mind2web: Towards a generalist agent for the web. In *Advances in Neural Information Processing Systems*, 2023. URL https://api.semanticscholar.org/CorpusID:259129428.

Aditya Dhodapkar and Farhaan Pishori. Safetydrift: Predicting when ai agents cross the line before they actually do. *ArXiv*, abs/2603.27148, 2026. URL https://api.semanticscholar.org/CorpusID:286961420.

Hang Ding, Peidong Liu, Junqiao Wang, Ziwei Ji, Meng Cao, Rongzhao Zhang, Lynn Ai, Eric Yang, Tianyu Shi, and Lei Yu. Dynaweb: Model-based reinforcement learning of web agents. *ArXiv*, abs/2601.22149, 2026. URL https://api.semanticscholar.org/CorpusID:285140906.

Jingtao Ding, Yunke Zhang, Yu Shang, Jie Feng, Yuheng Zhang, Zefang Zong, Yuan Yuan, Hongyuan Su, Nian Li, Jinghua Piao, Yucheng Deng, Nicholas Sukiennik, Chen Gao, Fengli Xu, and Yong Li. Understanding world or predicting future? a comprehensive survey of world models. *ACM Computing Surveys*, 58:1 – 38, 2025. URL https://api.semanticscholar.org/CorpusID:274192171.

Jiahua Dong, Qi Lyu, Baichen Liu, Xudong Wang, Wenqi Liang, Duzhen Zhang, Jiahang Tu, Hongliu Li, Hanbin Zhao, Henghui Ding, Yulun Zhang, Zhi Han, Nicu Sebe, Fahad Shahbaz Khan, Salman Khan, Mubarak Shah, Philip Torr, Ming-Hsuan Yang, and Dacheng Tao. Learning to model the world: A survey of world models in artificial intelligence. *Preprints.org*, 2026.

Alexey Dosovitskiy, Germán Ros, Felipe Codevilla, Antonio M. López, and Vladlen Koltun. Carla: An open urban driving simulator. In *Conference on Robot Learning*, 2017. URL https://api.semanticscholar.org/CorpusID:5550767.

Benjamin Ellis, Jonathan Cook, Skander Moalla, Mikayel Samvelyan, Mingfei Sun, Anuj Mahajan, Jakob N. Foerster, and Shimon Whiteson. Smacv2: An improved benchmark for cooperative multi-agent reinforcement learning. In *Advances in Neural Information Processing Systems*, 2023. URL https://api.semanticscholar.org/CorpusID:254685791.

FAIR CodeGen team, Jade Copet, Quentin Carbonneaux, Gal Cohen, Jonas Gehring, Jacob Kahn, Jannik Kossen, Felix Kreuk, Emily McMilin, Michel Meyer, Yuxiang Wei, David Zhang, Kunhao Zheng, Jordi Armengol-Estapé, Pedram Bashiri, Maximilian Beck, Pierre Chambon, Abhishek Charnalia, Chris Cummins, Juliette Decugis, Zacharias V. Fisches, François Fleuret, Fabian Gloeckle, Alex Gu, Michael Hassid, Daniel Haziza, Badr Youbi Idrissi, Christian Keller, Rahul Kindi, Hugh Leather, Gallil Maimon, Aram Markosyan, Francisco Massa, Pierre-Emmanuel Mazaré, Vegard Mella, Naila Murray, Keyur Muzumdar, Peter O'Hearn, Matteo Pagliardini, Dmitrii Pedchenko, Tal Remez, Volker Seeker, Marco

Selvi, Oren Sultan, Sida Wang, Luca Wehrstedt, Ori Yoran, Lingming Zhang, Taco Cohen, Yossi Adi, and Gabriel Synnaeve. Cwm: An open-weights llm for research on code generation with world models. *ArXiv*, abs/2510.02387, 2025. URL https://api.semanticscholar.org/CorpusID:281830152.

Linxi (Jim) Fan, Guanzhi Wang, Yunfan Jiang, Ajay Mandlekar, Yuncong Yang, Haoyi Zhu, Andy Tang, De-An Huang, Yuke Zhu, and Anima Anandkumar. Minedojo: Building open-ended embodied agents with internet-scale knowledge. In *Advances in Neural Information Processing Systems*, 2022. URL https://api.semanticscholar.org/CorpusID:249848263.

Tianqing Fang, Hongming Zhang, Zhisong Zhang, Kaixin Ma, Wenhao Yu, Haitao Mi, and Dong Yu. Webevolver: Enhancing web agent self-improvement with coevolving world model. In *Conference on Empirical Methods in Natural Language Processing*, 2025. URL https://api.semanticscholar.org/CorpusID:278208004.

Jichen Feng, Yifan Zhang, Chenggong Zhang, Yifu Lu, Shilong Liu, and Mengdi Wang. Web world models. *ArXiv*, abs/2512.23676, 2025a. URL https://api.semanticscholar.org/CorpusID:284311145.

Tongtong Feng, Xin Wang, Zekai Zhou, Ren Wang, Yuwei Zhan, Guangyao Li, Qing Li, and Wenwu Zhu. Evoagent: Self-evolving agent with continual world model for long-horizon tasks. In *International Conference on Learning Representations*, 2026. URL https://api.semanticscholar.org/CorpusID:276250457.

Tuo Feng, Wenguan Wang, and Yi Yang. A survey of world models for autonomous driving. *ArXiv*, abs/2501.11260, 2025b. URL https://api.semanticscholar.org/CorpusID:275757094.

Yifei Gao, Junhong Ye, Jiaqi Wang, and Jitao Sang. Websynthesis: World-model-guided mcts for efficient webui-trajectory synthesis. *ArXiv*, abs/2507.04370, 2025. URL https://api.semanticscholar.org/CorpusID:280068297.

Zhiqi Ge, Hongzhe Huang, Mingze Zhou, Juncheng Li, Guoming Wang, Siliang Tang, and Yueting Zhuang. Worldgpt: Empowering llm as multimodal world model. In *Proceedings of the 32nd ACM International Conference on Multimedia*, 2024. URL https://api.semanticscholar.org/CorpusID:269449620.

Yu Gu, Kai Zhang, Yuting Ning, Boyuan Zheng, Boyu Gou, Tianci Xue, Cheng Chang, Sanjari Srivastava, Yanan Xie, Peng Qi, Huan Sun, and Yu Su. Is your llm secretly a world model of the internet? model-based planning for web agents. *Transactions on Machine Learning Research*, 2025. URL https://api.semanticscholar.org/CorpusID:273963078.

Yiming Guan, Rui Yu, John Zhang, Lu Wang, Chaoyun Zhang, Liqun Li, Bo Qiao, Si Qin, He Huang, Fangkai Yang, Pu Zhao, Lukas Wutschitz, Samuel Kessler, Huseyin A Inan, Robert Sim, Saravan Rajmohan, Qingwei Lin, and Dongmei Zhang. Computer-using world model. *ArXiv*, abs/2602.17365, 2026. URL https://api.semanticscholar.org/CorpusID:285787857.

Lakshya Gupta, Litao Li, Yizhe Liu, Sriram Ganapathi Subramanian, Kaheer Suleman, Zichen Zhang, Haoye Lu, and Sumit Pasupalak. World of workflows: a benchmark for bringing world models to enterprise systems. *ArXiv*, abs/2601.22130, 2026. URL https://api.semanticscholar.org/CorpusID:285139845.

William H. Guss, Brandon Houghton, Nicholay Topin, Phillip Wang, Cayden R. Codel, Manuela M. Veloso, and Ruslan Salakhutdinov. Minerl: A large-scale dataset of minecraft demonstrations. In *International Joint Conference on Artificial Intelligence*, 2019. URL https://api.semanticscholar.org/CorpusID:199000710.

David Ha and Jürgen Schmidhuber. World models. In *Advances in Neural Information Processing Systems*, 2018. URL https://api.semanticscholar.org/CorpusID:4807711.

Danijar Hafner. Benchmarking the spectrum of agent capabilities. In *International Conference on Learning Representations*, 2022. URL https://api.semanticscholar.org/CorpusID:237504552.

Danijar Hafner, Timothy Lillicrap, Ian Fischer, Ruben Villegas, David Ha, Honglak Lee, and James Davidson. Learning latent dynamics for planning from pixels. In *International Conference on Machine Learning*, 2019. URL https://api.semanticscholar.org/CorpusID:53280207.

Danijar Hafner, Timothy Lillicrap, Jimmy Ba, and Mohammad Norouzi. Dream to control: Learning behaviors by latent imagination. In *International Conference on Learning Representations*, 2020. URL https://api.semanticscholar.org/CorpusID:208547755.

Danijar Hafner, Jurgis Pasukonis, Jimmy Ba, and Timothy Lillicrap. Mastering diverse control tasks through world models. *Nature*, 640:647 – 653, 2025. URL https://api.semanticscholar.org/CorpusID:277508993.

Bingguang Hao, Zengzhuang Xu, Yuntao Wen, Xinyi Xu, Yang Liu, Tong Zhao, Maolin Wang, Long Chen, Dong Wang, Yicheng Chen, Cunyin Peng, Xiangyu Zhao, Chenyi Zhuang, and Ji Zhang. From failure to mastery: Generating hard samples for tool-use agents. *ArXiv*, abs/2601.01498, 2026. URL https://api.semanticscholar.org/CorpusID:284487609.

Shibo Hao, Yi Gu, Haodi Ma, Joshua Jiahua Hong, Zhen Wang, Daisy Zhe Wang, and Zhiting Hu. Reasoning with language model is planning with world model. In *Conference on Empirical Methods in Natural Language Processing*, 2023. URL https://api.semanticscholar.org/CorpusID:258865812.

Dan Hendrycks, Steven Basart, Saurav Kadavath, Mantas Mazeika, Akul Arora, Ethan Guo, Collin Burns, Samir Puranik, Horace He, Dawn Xiaodong Song, and Jacob Steinhardt. Measuring coding challenge competence with apps. In *Advances in Neural Information Processing Systems*, 2021. URL https://api.semanticscholar.org/CorpusID:234790100.

Martin Heusel, Hubert Ramsauer, Thomas Unterthiner, Bernhard Nessler, and Sepp Hochreiter. Gans trained by a two time-scale update rule converge to a local nash equilibrium. In *Advances in Neural Information Processing Systems*, 2017. URL https://api.semanticscholar.org/CorpusID:326772.

Zhiting Hu and Tianmin Shu. Language models, agent models, and world models: The law for machine reasoning and planning. *ArXiv*, abs/2312.05230, 2023. URL https://api.semanticscholar.org/CorpusID:266149622.

Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. Swe-bench: Can language models resolve real-world github issues? In *International Conference on Learning Representations*, 2024. URL https://api.semanticscholar.org/CorpusID:263829697.

Raghav Kapoor, Yash Parag Butala, Melisa Russak, Jing Yu Koh, Kiran Kamble, Waseem Alshikh, and Ruslan Salakhutdinov. Omniact: A dataset and benchmark for enabling multimodal generalist autonomous agents for desktop and web. In *European Conference on Computer Vision*, 2024. URL https://api.semanticscholar.org/CorpusID:268031860.

Denis Kocetkov, Raymond Li, Loubna Ben Allal, Jia Li, Chenghao Mou, Carlos Muñoz Ferrandis, Yacine Jernite, Margaret Mitchell, Sean Hughes, Thomas Wolf, Dzmitry Bahdanau, Leandro von Werra, and Harm de Vries. The stack: 3 tb of permissively licensed source code. *Transactions on Machine Learning Research*, 2023. URL https://api.semanticscholar.org/CorpusID:254044610.

Jing Yu Koh, Robert Lo, Lawrence Jang, Vikram Duvvur, Ming Lim, Po-Yu Huang, Graham Neubig, Shuyan Zhou, Russ Salakhutdinov, and Daniel Fried. Visualwebarena: Evaluating multimodal agents on realistic visual web tasks. In *Annual Meeting of the Association for Computational Linguistics*, 2024. URL https://api.semanticscholar.org/CorpusID:267199749.

Woosung Koh, Sungjun Han, Segyu Lee, Se-Young Yun, and Jamin Shin. Generative visual code mobile world models. *ArXiv*, abs/2602.01576, 2026. URL https://api.semanticscholar.org/CorpusID:285270039.

Lingdong Kong, Wesley Yang, Jianbiao Mei, Youquan Liu, Ao Liang, Dekai Zhu, Dongyue Lu, Wei Yin, Xiaotao Hu, Mingkai Jia, Junyuan Deng, Kaiwen Zhang, Yang Wu, Tianyi Yan, Shenyuan Gao, Song Wang, Linfeng Li, Liang Pan, Yong Liu, Jianke Zhu, Wei Tsang Ooi, Steven C. H. Hoi, and Ziwei Liu. 3d and 4d world modeling: A survey. *ArXiv*, abs/2509.07996, 2025a. URL https://api.semanticscholar.org/CorpusID:281243924.

Quyu Kong, Xu Zhang, Zhenyu Yang, Nolan Gao, Chen Liu, Panrong Tong, Chenglin Cai, Hanzhang Zhou, Jianan Zhang, Liangyu Chen, Zhidan Liu, Steven Hoi, and Yue Wang. Mobileworld: Benchmarking autonomous mobile agents in agent-user interactive and mcp-augmented environments. *ArXiv*, abs/2512.19432, 2025b. URL https://api.semanticscholar.org/CorpusID:284078735.

Heinrich Kuttler, Nantas Nardelli, Alexander H. Miller, Roberta Raileanu, Marco Selvatici, Edward Grefenstette, and Tim Rocktäschel. The nethack learning environment. In *Advances in Neural Information Processing Systems*, 2020. URL https://api.semanticscholar.org/CorpusID:220042384.

Nathan Lambert, Brandon Amos, Omry Yadan, and Roberto Calandra. Objective mismatch in model-based reinforcement learning. In *Learning for Dynamics and Control*, pp. 761–770, 2020.

Haodong Li, Jingwei Wu, Quan Sun, Guopeng Li, Juanxi Tian, Huanyu Zhang, Yanlin Lai, Ruichuan An, Hong Peng, Yuhong Dai, Chenxi Li, Chunmei Qing, Jia Wang, Ziyang Meng, Zheng Ge, Xiangyu Zhang, and Daxin Jiang. Gebench: Benchmarking image generation models as gui environments. *ArXiv*, abs/2602.09007, 2026a. URL https://api.semanticscholar.org/CorpusID:285452406.

Minghao Li, Yingxiu Zhao, Bowen Yu, Feifan Song, Hangyu Li, Haiyang Yu, Zhoujun Li, Fei Huang, and Yongbin Li. Api-bank: A comprehensive benchmark for tool-augmented llms. In *Conference on Empirical Methods in Natural Language Processing*, 2023a. URL https://api.semanticscholar.org/CorpusID:258179056.

Raymond Li, Loubna Ben Allal, Yangtian Zi, Niklas Muennighoff, Denis Kocetkov, Chenghao Mou, Marc Marone, Christopher Akiki, Jia Li, Jenny Chim, Qian Liu, Evgenii Zheltonozhskii, Terry Yue Zhuo, Thomas Wang, Olivier Dehaene, Mishig Davaadorj, Joel Lamy-Poirier, João Monteiro, Oleh Shliazhko, Nicolas Gontier, Nicholas Meade, Armel Zebaze, Ming-Ho Yee, Logesh Kumar Umapathi, Jian Zhu, Benjamin Lipkin, Muhtasham Oblokulov, Zhiruo Wang, Rudra Murthy, Jason Stillerman, Siva Sankalp Patel, Dmitry Abulkhanov, Marco Zocca, Manan Dey, Zhihan Zhang, Nour Fahmy, Urvashi Bhattacharyya, Wenhao Yu, Swayam Singh, Sasha Luccioni, Paulo Villegas, Maxim Kunakov, Fedor Zhdanov, Manuel Romero, Tony Lee, Nadav Timor, Jennifer Ding, Claire Schlesinger, Hailey Schoelkopf, Jan Ebert, Tri Dao, Mayank Mishra, Alex Gu, Jennifer Robinson, Carolyn Jane Anderson, Brendan Dolan-Gavitt, Danish Contractor, Siva Reddy, Daniel Fried, Dzmitry Bahdanau, Yacine Jernite, Carlos Muñoz Ferrandis, Sean Hughes, Thomas Wolf, Arjun Guha, Leandro von Werra, and Harm de Vries. Starcoder: may the source be with you! *Transactions on Machine Learning Research*, 2023b. URL https://api.semanticscholar.org/CorpusID:258588247.

Shufan Li, Konstantinos Kallidromitis, Akash Gokul, Yusuke Kato, Kazuki Kozuka, and Aditya Grover. Mobileworldbench: Towards semantic world modeling for mobile agents. In *Conference on Language Modeling*, 2025a. URL https://api.semanticscholar.org/CorpusID:283909511.

Wei Li, William Bishop, Alice Li, Chris Rawles, Folawiyo Campbell-Ajala, Divya Tyamagundlu, and Oriana Riva. On the effects of data scale on ui control agents. In *Advances in Neural Information Processing Systems*, 2024. URL https://api.semanticscholar.org/CorpusID:270285816.

Xinqing Li, Xin He, Le Zhang, Min Wu, Xiaoli Li, and Yun Liu. A comprehensive survey on world models for embodied ai. *ArXiv*, abs/2510.16732, 2025b. URL https://api.semanticscholar.org/CorpusID:282210370.

Yixia Li, Hongru Wang, Jiahao Qiu, Zhenfei Yin, Dongdong Zhang, Cheng Qian, Zeping Li, Pony Ma, Guanhua Chen, and Heng Ji. From word to world: Can large language models be implicit text-based world models? In *Annual Meeting of the Association for Computational Linguistics*, 2026b. URL https://api.semanticscholar.org/CorpusID:284078241.

Yujia Li, David Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser, Rémi Leblond, Tom Eccles, James Keeling, Felix Gimeno, Agustin Dal Lago, Thomas Hubert, Peter Choy, Cyprien de Masson d'Autume, Igor Babuschkin, Xinyun Chen, Po-Sen Huang, Johannes Welbl, Sven Gowal, Alexey Cherepanov, James Molloy, Daniel Jaymin Mankowitz, Esme Sutherland Robson, Pushmeet Kohli, Nando de Freitas, Koray Kavukcuoglu, and Oriol Vinyals. Competition-level code generation with alphacode. *Science*, 378:1092 – 1097, 2022. URL https://api.semanticscholar.org/CorpusID:246527904.

Evan Zheran Liu, Kelvin Guu, Panupong Pasupat, Tianlin Shi, and Percy Liang. Reinforcement learning on web interfaces using workflow-guided exploration. In *International Conference on Learning Representations*, 2018. URL https://api.semanticscholar.org/CorpusID:3530344.

Xiao Liu, Bo Qin, Dongzhu Liang, Guang Dong, Hanyu Lai, Hanchen Zhang, Hanlin Zhao, Iat Long Iong, Jiadai Sun, Jiaqi Wang, Junjie Gao, Junjun Shan, Kangning Liu, Shudan Zhang, Shuntian Yao, Siyi Cheng, Wentao Yao, Wenyi Zhao, Xinghan Liu, Xinyi Liu, Xinying Chen, Xinyue Yang, Yang Yang, Yifan Xu, Yu Yang, Yujia Wang, Yulin Xu, Zehan Qi, Yuxiao Dong, and Jie Tang. Autoglm: Autonomous foundation agents for guis. *ArXiv*, abs/2411.00820, 2024a. URL https://api.semanticscholar.org/CorpusID:273811941.

Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, Shudan Zhang, Xiang Deng, Aohan Zeng, Zhengxiao Du, Chenhui Zhang, Sheng Shen, Tianjun Zhang, Yu Su, Huan Sun, Minlie Huang, Yuxiao Dong, and Jie Tang. Agentbench: Evaluating llms as agents. In *International Conference on Learning Representations*, 2024b. URL https://api.semanticscholar.org/CorpusID:260682249.

Zhihan Liu, Hao Hu, Shenao Zhang, Hongyi Guo, Shuqi Ke, Boyi Liu, and Zhaoran Wang. Reason for future, act for now: A principled architecture for autonomous llm agents with provable sample efficiency. In *International Conference on Machine Learning*, 2024c. URL https://api.semanticscholar.org/CorpusID:263310943.

Jieyi Long. Large language model guided tree-of-thought. *ArXiv*, abs/2305.08291, 2023. URL https://api.semanticscholar.org/CorpusID:258686311.

Xiaoxiao Long, Qingrui Zhao, Kaiwen Zhang, Zihao Zhang, Dingrui Wang, Yumeng Liu, Zhengjie Shu, Yi Lu, Shouzheng Wang, Xinzhe Wei, Wei Li, Wei Yin, Yao Yao, Jia Pan, Qiu Shen, Ruigang Yang, Xun Cao, and Qionghai Dai. A survey: Learning embodied intelligence from physical simulators and world models. *ArXiv*, abs/2507.00917, 2025. URL https://api.semanticscholar.org/CorpusID:280137292.

Shuai Lu, Daya Guo, Shuo Ren, Junjie Huang, Alexey Svyatkovskiy, Ambrosio Blanco, Colin B. Clement, Dawn Drain, Daxin Jiang, Duyu Tang, Ge Li, Lidong Zhou, Linjun Shou, Long Zhou, Michele Tufano, Ming Gong, Ming Zhou, Nan Duan, Neel Sundaresan, Shao Kun Deng, Shengyu Fu, and Shujie Liu. Codexglue: A machine learning benchmark dataset for code understanding and generation. In *Advances in Neural Information Processing Systems*, 2021. URL https://api.semanticscholar.org/CorpusID:231855531.

Xing Han Lù, Zdeněk Kasner, and Siva Reddy. Weblinx: Real-world website navigation with multi-turn dialogue. In *International Conference on Machine Learning*, 2024. URL https://api.semanticscholar.org/CorpusID:267547883.

Dezhao Luo, Bohan Tang, Kang Li, Georgios Papoudakis, Jifei Song, Shaogang Gong, Jianye Hao, Jun Wang, and Kun Shao. Vimo: A generative visual gui world model for app agents. In *International Conference on Learning Representations*, 2025a. URL https://api.semanticscholar.org/CorpusID:277955457.

Ziyang Luo, Zhiqi Shen, Wenzhuo Yang, Zirui Zhao, Prathyusha Jwalapuram, Amrita Saha, Doyen Sahoo, Silvio Savarese, Caiming Xiong, and Junnan Li. Mcp-universe: Benchmarking large language models with real-world model context protocol servers. *ArXiv*, abs/2508.14704, 2025b. URL https://api.semanticscholar.org/CorpusID:280691759.

Amir massoud Farahmand. Iterative value-aware model learning. In *Advances in Neural Information Processing Systems*, volume 31, 2018.

Kai Mei, Jiang Guo, Shuaichen Chang, Mingwen Dong, Dongkyu Lee, Xing Niu, and Jiarong Jiang. R-wom: Retrieval-augmented world model for computer-use agents. In *International Conference on Learning Representations*, 2026. URL https://api.semanticscholar.org/CorpusID:282064561.

Vincent Micheli, Eloi Alonso, and François Fleuret. Transformers are sample-efficient world models. In *International Conference on Learning Representations*, 2023. URL https://api.semanticscholar.org/CorpusID:251979354.

Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A. Rusu, Joel Veness, Marc G. Bellemare, Alex Graves, Martin A. Riedmiller, Andreas Kirkeby Fidjeland, Georg Ostrovski, Stig Petersen, Charlie Beattie, Amir Sadik, Ioannis Antonoglou, Helen King, Dharshan Kumaran, Daan Wierstra, Shane Legg, and Demis Hassabis. Human-level control through deep reinforcement learning. *Nature*, 518:529–533, 2015. URL https://api.semanticscholar.org/CorpusID:205242740.

Jian Mu, Chaoyun Zhang, Chiming Ni, Lu Wang, Bo Qiao, Kartik Mathur, Qianhui Wu, Yuhang Xie, Xiaojun Ma, Mengyu Zhou, Si Qin, Liqun Li, Yu Kang, Minghua Ma, Qingwei Lin, Saravan Rajmohan, and Dongmei Zhang. GUI-360°: A comprehensive dataset and benchmark for computer-using agents. In *International Conference on Learning Representations*, 2026. URL https://api.semanticscholar.org/CorpusID:282812257.

Shishir G. Patil, Huanzhi Mao, Fanjia Yan, Charlie Cheng-Jie Ji, Vishnu Suresh, Ion Stoica, and Joseph Gonzalez. The berkeley function calling leaderboard (bfcl): From tool use to agentic evaluation of large language models. In *International Conference on Machine Learning*, 2025. URL https://api.semanticscholar.org/CorpusID:283567780.

Cheng Qian, Emre Can Acikgoz, Bingxuan Li, Xiusi Chen, Yuji Zhang, Bingxiang He, Qinyu Luo, Dilek Hakkani-Tur, Gokhan Tur, Yunzhu Li, and Heng Ji. Current agents fail to leverage world model as tool for foresight. In *Annual Meeting of the Association for Computational Linguistics*, 2026. URL https://api.semanticscholar.org/CorpusID:284532584.

Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, Sihan Zhao, Lauren Hong, Runchu Tian, Ruobing Xie, Jie Zhou, Mark Gerstein, Dahai Li, Zhiyuan Liu, and Maosong Sun. Toolllm: Facilitating large language models to master 16000+ real-world apis. In *International Conference on Learning Representations*, 2024. URL https://api.semanticscholar.org/CorpusID:260334759.

Yujia Qin, Yining Ye, Junjie Fang, Haoming Wang, Shihao Liang, Shizuo Tian, Junda Zhang, Jiahao Li, Yunxin Li, Shijue Huang, Wanjun Zhong, Kuanye Li, Jiale Yang, Yu Miao, Woyu Lin, Longxiang Liu, Xu Jiang, Qianli Ma, Jingyu Li, Xiaojun Xiao, Kai Cai, Chuang Li, Yaowei Zheng, Chaolin Jin, Chen Li, Xiao Zhou, Minchao Wang, Haoli Chen, Zhaojian Li, Haihua Yang, Haifeng Liu, Feng Lin, Tao Peng, Xin Liu, and Guang Shi. Ui-tars: Pioneering automated gui interaction with native agents. *ArXiv*, abs/2501.12326, 2025. URL https://api.semanticscholar.org/CorpusID:275788034.

Christopher Rawles, Alice Li, Daniel Rodriguez, Oriana Riva, and Timothy Lillicrap. Android in the wild: A large-scale dataset for android device control. In *Advances in Neural Information Processing Systems*, 2023. URL https://api.semanticscholar.org/CorpusID:259983067.

Christopher Rawles, Sarah Clinckemaillie, Yifan Chang, Jonathan Waltz, Gabrielle Lau, Marybeth Fair, Alice Li, William Bishop, Wei Li, Folawiyo Campbell-Ajala, Daniel Toyama, Robert Berry, Divya Tyamagundlu, Timothy Lillicrap, and Oriana Riva. Androidworld: A dynamic benchmarking environment for autonomous agents. In *International Conference on Learning Representations*, 2025. URL https://api.semanticscholar.org/CorpusID:269982433.

Soham Ray, Keshav Dhandhania, Victor Barres, and Karthik Narasimhan. τ-voice: Benchmarking full-duplex voice agents on real-world domains. *ArXiv*, abs/2603.13686, 2026. URL https://api.semanticscholar.org/CorpusID:286568451.

Zhenzhen Ren, Xinpeng Zhang, Zhenxing Qian, Yan Gao, Yu Shi, Shuxin Zheng, and Jiyan He. Gtm: Simulating the world of tools for ai agents. *ArXiv*, abs/2512.04535, 2025. URL https://api.semanticscholar.org/CorpusID:283556867.

Luke Rivard, Sun Sun, Hongyu Guo, Wenhu Chen, and Yuntian Deng. Neuralos: Towards simulating operating systems via neural generative models. In *International Conference on Learning Representations*, 2026. URL https://api.semanticscholar.org/CorpusID:280282662.

Baptiste Rozière, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Romain Sauvestre, Tal Remez, Jérémy Rapin, Artyom Kozhevnikov, Ivan Evtimov, Joanna Bitton, Manish Bhatt, Cristian Canton Ferrer, Aaron Grattafiori, Wenhan Xiong, Alexandre Défossez, Jade Copet, Faisal Azhar, Hugo Touvron, Louis Martin, Nicolas Usunier, Thomas Scialom, and Gabriel Synnaeve. Code llama: Open foundation models for code. In *Advances in Neural Information Processing Systems*, 2024. URL https://api.semanticscholar.org/CorpusID:261100919.

Mikayel Samvelyan, Tabish Rashid, Christian Schroeder de Witt, Gregory Farquhar, Nantas Nardelli, Tim G. J. Rudner, Chia-Man Hung, Philip H. S. Torr, Jakob Foerster, and Shimon Whiteson. The starcraft multi-agent challenge. In *International Conference on Autonomous Agents and Multi-Agent Systems*, 2019. URL https://api.semanticscholar.org/CorpusID:60440549.

Mikayel Samvelyan, Robert Kirk, Vitaly Kurin, Jack Parker-Holder, Minqi Jiang, Eric Hambro, Fabio Petroni, Heinrich Küttler, Edward Grefenstette, and Tim Rocktäschel. Minihack the planet: A sandbox for open-ended reinforcement learning research. In *Advances in Neural Information Processing Systems*, 2021. URL https://api.semanticscholar.org/CorpusID:237274213.

Julian Schrittwieser, Ioannis Antonoglou, Thomas Hubert, Karen Simonyan, Laurent Sifre, Simon Schmitt, Arthur Guez, Edward Lockhart, Demis Hassabis, Thore Graepel, Timothy Lillicrap, and David Silver. Mastering atari, go, chess and shogi by planning with a learned model. *Nature*, 588:604 – 609, 2020. URL https://api.semanticscholar.org/CorpusID:208158225.

Zhouzhou Shen, Xueyu Hu, Xiyun Li, Tianqing Fang, Juncheng Li, and Shengyu Zhang. World-model-augmented web agents with action correction. *ArXiv*, abs/2602.15384, 2026. URL https://api.semanticscholar.org/CorpusID:285659218.

Tianlin Shi, Andrej Karpathy, Linxi Fan, Jonathan Hernandez, and Percy Liang. World of bits: An open-domain platform for web-based agents. In *International Conference on Machine Learning*, 2017. URL https://api.semanticscholar.org/CorpusID:34953552.

Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems*, 2023.

Mohit Shridhar, Xingdi Yuan, Marc-Alexandre Côté, Yonatan Bisk, Adam Trischler, and Matthew J. Hausknecht. Alfworld: Aligning text and embodied environments for interactive learning. In *International Conference on Learning Representations*, 2021. URL https://api.semanticscholar.org/CorpusID:222208810.

David Silver, Aja Huang, Chris J. Maddison, Arthur Guez, L. Sifre, George van den Driessche, Julian Schrittwieser, Ioannis Antonoglou, Vedavyas Panneershelvam, Marc Lanctot, Sander Dieleman, Dominik Grewe, John Nham, Nal Kalchbrenner, Ilya Sutskever, Timothy P. Lillicrap, Madeleine Leach, Koray Kavukcuoglu, Thore Graepel, and Demis Hassabis. Mastering the game of go with deep neural networks and tree search. *Nature*, 529:484–489, 2016. URL https://api.semanticscholar.org/CorpusID:515925.

Jianwen Sun, Yukang Feng, Kaining Ying, Chuanhao Li, Zizhen Li, Fanrui Zhang, Jiaxin Ai, Yifan Chang, Yu Dai, Yifei Huang, and Kaipeng Zhang. World craft: Agentic framework to create visualizable worlds via text. *ArXiv*, abs/2601.09150, 2026a. URL https://api.semanticscholar.org/CorpusID:284717831.

Shuang Sun, Huatong Song, Lisheng Huang, Jinhao Jiang, Ran Le, Zhihao Lv, Zongchao Chen, Yiwen Hu, Wenyang Luo, Wayne Xin Zhao, Yang Song, Hongteng Xu, Tao Zhang, and Ji-Rong Wen. Swe-world: Building software engineering agents in docker-free environments. *ArXiv*, abs/2602.03419, 2026b. URL https://api.semanticscholar.org/CorpusID:285275006.

Wentao Tan, Lei Zhu, Bowen Wang, Enci Xie, Baixu Ji, Zengrong Lin, Wenjie Yang, Jingjing Li, and Heng Tao Shen. Towards generalist embodied ai: A survey on world models for vla agents. *TechRxiv Preprints*, 2026.

Hao Tang, Darren Key, and Kevin Ellis. Worldcoder, a model-based llm agent: Building world models by writing code and interacting with the environment. In *Advances in Neural Information Processing Systems*, 2024. URL https://api.semanticscholar.org/CorpusID:267751283.

Qiaoyu Tang, Ziliang Deng, Hongyu Lin, Xianpei Han, Qiao Liang, Boxi Cao, and Le Sun. Toolalpaca: Generalized tool learning for language models with 3000 simulated cases. *ArXiv*, abs/2306.05301, 2023. URL https://api.semanticscholar.org/CorpusID:259108190.

Venus Team, Changlong Gao, Zhangxuan Gu, Yulin Liu, Xinyu Qiu, Shuheng Shen, Yue Wen, Tianyu Xia, Zhenyu Xu, Zhengwen Zeng, Beitong Zhou, Xingran Zhou, Weizhi Chen, Sunhao Dai, Jingya Dou, Yichen Gong, Yuan Guo, Zhenlin Guo, Feng Li, Qian Li, Jinzhen Lin, Yuqi Zhou, Linchao Zhu, Liang Chen, Zhenyu Guo, Changhua Meng, and Weiqiang Wang. Ui-venus-1.5 technical report. *ArXiv*, abs/2602.09082, 2026. URL https://api.semanticscholar.org/CorpusID:285462282.

Emanuel Todorov, Tom Erez, and Yuval Tassa. Mujoco: A physics engine for model-based control. In *IEEE/RSJ International Conference on Intelligent Robots and Systems*, pp. 5026–5033, 2012. URL https://api.semanticscholar.org/CorpusID:5230692.

Sifan Tu, Xin Zhou, Dingkang Liang, Xingyu Jiang, Yumeng Zhang, Xiaofan Li, and Xiang Bai. The role of world models in shaping autonomous driving: A comprehensive survey. *ArXiv*, abs/2502.10498, 2025. URL https://api.semanticscholar.org/CorpusID:276408158.

Thomas Unterthiner, Sjoerd van Steenkiste, Karol Kurach, Raphaël Marinier, Marcin Michalski, and Sylvain Gelly. Towards accurate generative models of video: A new metric & challenges. *ArXiv*, abs/1812.01717, 2018. URL https://api.semanticscholar.org/CorpusID:54458806.

Oriol Vinyals, Igor Babuschkin, Wojciech M. Czarnecki, Michaël Mathieu, Andrew Joseph Dudzik, Junyoung Chung, David Choi, Richard Powell, Timo Ewalds, Petko Georgiev, Junhyuk Oh, Dan Horgan, Manuel Kroiss, Ivo Danihelka, Aja Huang, L. Sifre, Trevor Cai, John P. Agapiou, Max Jaderberg, Alexander Sasha Vezhnevets, Rémi Leblond, Tobias Pohlen, Valentin Dalibard, David Budden, Yury Sulsky, James Molloy, Tom Le Paine, Caglar Gulcehre, Ziyun Wang, Tobias Pfaff, Yuhuai Wu, Roman Ring, Dani Yogatama, Dario Wünsch, Katrina McKinney, Oliver Smith, Tom Schaul, Timothy P. Lillicrap, Koray Kavukcuoglu, Demis Hassabis, Chris Apps, and David Silver. Grandmaster level in starcraft ii using multi-agent reinforcement learning. *Nature*, 575:350 – 354, 2019. URL https://api.semanticscholar.org/CorpusID:204972004.

Kangrui Wang, Pingyue Zhang, Zihan Wang, Yaning Gao, Linjie Li, Qineng Wang, Hanyang Chen, Chi Wan, Yiping Lu, Zhengyuan Yang, Lijuan Wang, Ranjay Krishna, Jiajun Wu, Fei-Fei Li, Yejin Choi, and Manling Li. Vagen: Reinforcing world model reasoning for multi-turn vlm agents. In *Advances in Neural Information Processing Systems*, 2025a. URL https://api.semanticscholar.org/CorpusID:282210682.

Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, Jiakai Tang, Xu Chen, Yankai Lin, Wayne Xin Zhao, Zhewei Wei, and Ji-Rong Wen. A survey on large language model based autonomous agents. *Frontiers of Computer Science*, 18, 2024. URL https://api.semanticscholar.org/CorpusID:261064713.

Ruoyao Wang, Peter Jansen, Marc-Alexandre Côté, and Prithviraj Ammanabrolu. Scienceworld: Is your agent smarter than a 5th grader? In *Conference on Empirical Methods in Natural Language Processing*, 2022. URL https://api.semanticscholar.org/CorpusID:247451124.

Yiming Wang, Da Yin, Yuedong Cui, Ruichen Zheng, Zhiqian Li, Zongyu Lin, Di Wu, Xueqing Wu, Chenchen Ye, Yu Zhou, and Kai-Wei Chang. Llms as scalable, general-purpose simulators for evolving digital agent training. *ArXiv*, abs/2510.14969, 2025b. URL https://api.semanticscholar.org/CorpusID:282138614.

Zhaoyang Wang, Canwen Xu, Boyi Liu, Yite Wang, Siwei Han, Zhewei Yao, Huaxiu Yao, and Yuxiong He. Agent world model: Infinity synthetic environments for agentic reinforcement learning. *ArXiv*, abs/2602.10090, 2026. URL https://api.semanticscholar.org/CorpusID:285463177.

Zhou Wang, Alan Conrad Bovik, Hamid R. Sheikh, and Eero P. Simoncelli. Image quality assessment: from error visibility to structural similarity. *IEEE Transactions on Image Processing*, 13:600–612, 2004. URL https://api.semanticscholar.org/CorpusID:207761262.

Zihao Wang, Xujing Li, Yining Ye, Junjie Fang, Haoming Wang, Longxiang Liu, Shihao Liang, Junting Lu, Zhiyong Wu, Jiazhan Feng, Wanjun Zhong, Zili Li, Yu Wang, Yu Miao, Bo Zhou, Yuanfan Li, Hao Wang, Zhongkai Zhao, Faming Wu, Zhengxuan Jiang, Weihao Tan, Heyuan Yao, Shi Yan, Xiangyang Li, Yitao Liang, Yujia Qin, and Guang Shi. Game-tars: Pretrained foundation models for scalable generalist multimodal game agents. *ArXiv*, abs/2510.23691, 2025c. URL https://api.semanticscholar.org/CorpusID:282401323.

Zhiheng Xi, Yiwen Ding, Wenxiang Chen, Boyang Hong, Honglin Guo, Junzhe Wang, Xin Guo, Dingwen Yang, Chenyang Liao, Wei He, Songyang Gao, Lu Chen, Rui Zheng, Yicheng Zou, Tao Gui, Qi Zhang, Xipeng Qiu, Xuanjing Huang, Zuxuan Wu, and Yu-Gang Jiang. Agentgym: Evaluating and training large language model-based agents across diverse environments. In *Annual Meeting of the Association for Computational Linguistics*, 2025. URL https://api.semanticscholar.org/CorpusID:270285866.

Jiannan Xiang, Yun Zhu, Lei Shu, Maria Wang, Lijun Yu, Gabriel Barcik, James Lyon, Srinivas Sunkara, and Jindong Chen. Uisim: An interactive image-based ui simulator for dynamic mobile environments. In *NeurIPS 2025 Workshop on Bridging Language, Agent, and World Models (LAW)*, 2025. URL https://api.semanticscholar.org/CorpusID:281658896.

Zikai Xiao, Jianhong Tu, Chuhang Zou, Yuxin Zuo, Zhi Li, Peng Wang, Bowen Yu, Fei Huang, Junyang Lin, and Zuozhu Liu. Webworld: A large-scale world model for web agent training. *ArXiv*, abs/2602.14721, 2026. URL https://api.semanticscholar.org/CorpusID:285616387.

Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh Jing Hua, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, Yitao Liu, Yiheng Xu, Shuyan Zhou, Silvio Savarese, Caiming Xiong, Victor Zhong, and Tao Yu. Osworld: Benchmarking multimodal agents for open-ended tasks in real computer environments. In *Advances in Neural Information Processing Systems*, 2024. URL https://api.semanticscholar.org/CorpusID:269042918.

Frank F. Xu, Yufan Song, Boxuan Li, Yuxuan Tang, Kritanjali Jain, Mengxue Bao, Zora Zhiruo Wang, Xuhui Zhou, Zhitong Guo, Murong Cao, Ming-Hsuan Yang, Hao Lu, Amaad Martin, Zhe Su, Leander Melroy Maben, Raj Mehta, Wayne Chi, Lawrence Jang, Yiqing Xie, Shuyan Zhou, and Graham Neubig. Theagentcompany: Benchmarking llm agents on consequential real world tasks. In *Advances in Neural Information Processing Systems*, 2025. URL https://api.semanticscholar.org/CorpusID:274822848.

John Yang, Akshara Prabhakar, Karthik Narasimhan, and Shunyu Yao. Intercode: Standardizing and benchmarking interactive coding with execution feedback. In *Advances in Neural Information Processing Systems*, 2023. URL https://api.semanticscholar.org/CorpusID:259262186.

John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. Swe-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems*, 2024. URL https://api.semanticscholar.org/CorpusID:270063685.

Shunyu Yao, Howard Chen, John Yang, and Karthik Narasimhan. Webshop: Towards scalable real-world web interaction with grounded language agents. In *Advances in Neural Information Processing Systems*, 2022. URL https://api.semanticscholar.org/CorpusID:250264533.

Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L. Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. In *Advances in Neural Information Processing Systems*, 2023a. URL https://api.semanticscholar.org/CorpusID:258762525.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023b. URL https://api.semanticscholar.org/CorpusID:252762395.

Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. τ-bench: A benchmark for tool-agent-user interaction in real-world domains. In *International Conference on Learning Representations*, 2025. URL https://api.semanticscholar.org/CorpusID:270562578.

Weirui Ye, Shaohuai Liu, Thanard Kurutach, Pieter Abbeel, and Yang Gao. Mastering atari games with limited data. In *Advances in Neural Information Processing Systems*, 2021. URL https://api.semanticscholar.org/CorpusID:240354728.

Xiao Yu, Baolin Peng, Ruize Xu, Yelong Shen, Pengcheng He, Suman Nath, Nikhil Singh, Jiangfeng Gao, and Zhou Yu. Reinforcement world model learning for llm-based agents. *ArXiv*, abs/2602.05842, 2026. URL https://api.semanticscholar.org/CorpusID:285303980.

Pengfei Zhang, Ying Cheng, Xiaofan Sun, Shijie Wang, Fengling Li, Lei Zhu, and Heng Tao Shen. A step toward world models: A survey on robotic manipulation. *ArXiv*, abs/2511.02097, 2025. URL https://api.semanticscholar.org/CorpusID:282749247.

Richard Zhang, Phillip Isola, Alexei A. Efros, Eli Shechtman, and Oliver Wang. The unreasonable effectiveness of deep features as a perceptual metric. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 586–595, 2018. URL https://api.semanticscholar.org/CorpusID:4766599.

Ruohan Zhang, Calen Walshe, Zhuode Liu, Lin Guan, Karl S. Muller, Jake A. Whritner, Luxin Zhang, Mary M. Hayhoe, and Dana H. Ballard. Atari-head: Atari human eye-tracking and demonstration dataset. In *Proceedings of the AAAI Conference on Artificial Intelligence*, pp. 6811–6820, 2020. URL https://api.semanticscholar.org/CorpusID:81982513.

Weipu Zhang, Gang Wang, Jian Sun, Yetian Yuan, and Gao Huang. Storm: Efficient stochastic transformer based world models for reinforcement learning. In *Advances in Neural Information Processing Systems*, 2023. URL https://api.semanticscholar.org/CorpusID:264145997.

Hongyu Zhao, Siyu Zhou, Haolin Yang, Zengyi Qin, and Tianyi Zhou. Neuro-symbolic synergy for interactive world modeling. *ArXiv*, abs/2602.10480, 2026. URL https://api.semanticscholar.org/CorpusID:285470906.

Yuhao Zheng, Li'an Zhong, Yi Wang, Rui Dai, Kaikui Liu, Xiangxiang Chu, Linyuan Lv, Philip Torr, and Kevin Qinghong Lin. Code2world: A gui world model via renderable code generation. *ArXiv*, abs/2602.09856, 2026. URL https://api.semanticscholar.org/CorpusID:285462615.

Andy Zhou, Kai Yan, Michal Shlapentokh-Rothman, Haohan Wang, and Yu-Xiong Wang. Language agent tree search unifies reasoning, acting, and planning in language models. In *International Conference on Machine Learning*, 2024a.

Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. Webarena: A realistic web environment for building autonomous agents. In *International Conference on Learning Representations*, 2024b. URL https://api.semanticscholar.org/CorpusID:260164780.

Siyu Zhou, Tianyi Zhou, Yijun Yang, Guodong Long, Deheng Ye, Jing Jiang, and Chengqi Zhang. Wall-e: World alignment by rule learning improves world model-based llm agents. *ArXiv*, abs/2410.07484, 2024c. URL https://api.semanticscholar.org/CorpusID:273233468.

Siyu Zhou, Tianyi Zhou, Yijun Yang, Guodong Long, Deheng Ye, Jing Jiang, and Chengqi Zhang. Wall-e: World alignment by neurosymbolic learning improves world model-based llm agents. In *Advances in Neural Information Processing Systems*, 2025. URL https://api.semanticscholar.org/CorpusID:277993974.

Zheng Zhu, Xiaofeng Wang, Wangbo Zhao, Chen Min, Bohan Li, Nianchen Deng, Min Dou, Yuqi Wang, Botian Shi, Kai Wang, Chi Zhang, Yang You, Zhaoxiang Zhang, Dawei Zhao, Liang Xiao, Jian Zhao, Jiwen Lu, and Guan Huang. Is sora a world simulator? a comprehensive survey on general world models and beyond. *ArXiv*, abs/2405.03520, 2024. URL https://api.semanticscholar.org/CorpusID:269604832.